



湖南大学
HUNAN UNIVERSITY

第十二章 单源最短路径

—— 湖南大学计算机学院 ——

目录

第一节

最短路径问题

第二节

松弛算法

第三节

有向无环图上的 最短路径计算

第四节

Dijkstra算法

第五节

Bellman-Ford算法

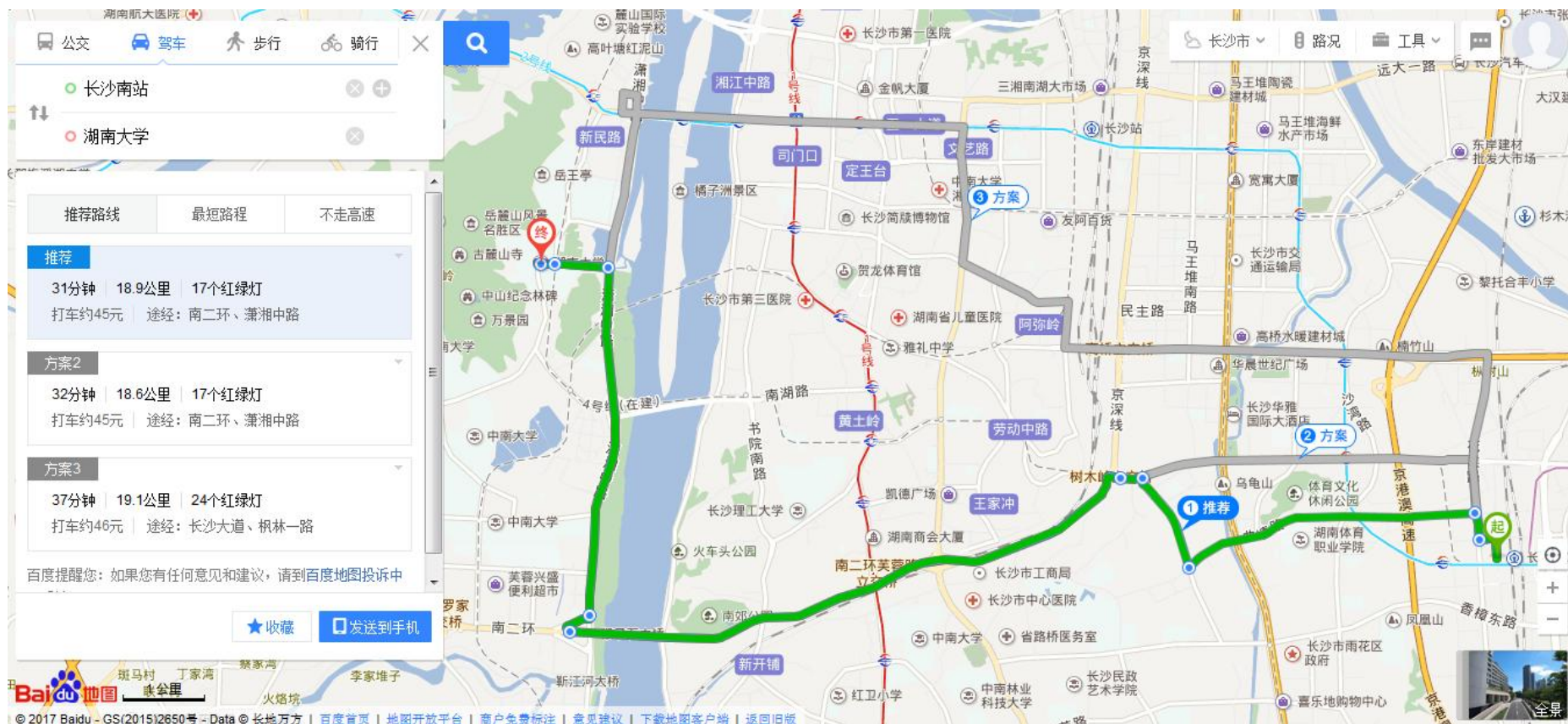
第六节

差分约束和最 短路径

第六节

路径计算前沿

最短路径问题——应用

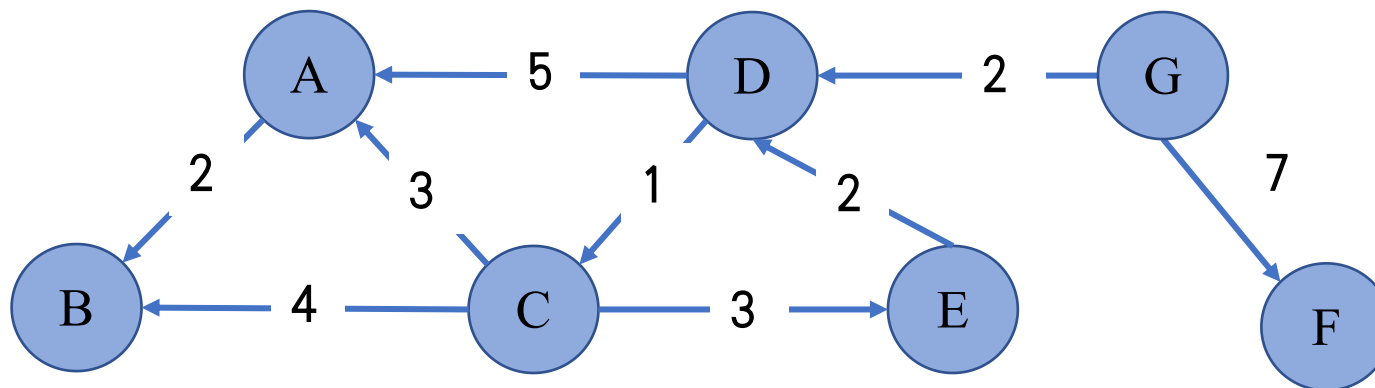


最短路径问题



带权图:

带权图可以表示成一个三元组 $G = \langle V, E, W \rangle$, 其中 V 是点集合, $E \subseteq V \times V$ 表示边集合, 其中每条边由 V 中两个点相连接所构成, $W: E \rightarrow R$ 将 E 中每条边 (u, v) 被赋上一个权重, 记为 $w(u, v)$

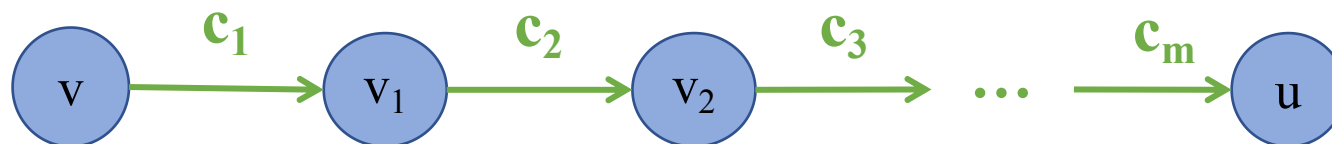


最短路径问题



最短路径:

考虑带权有向图，把一条路径（仅仅考虑简单路径）上所经边的权值之和定义为该路径的路径长度或称带权路径长度。



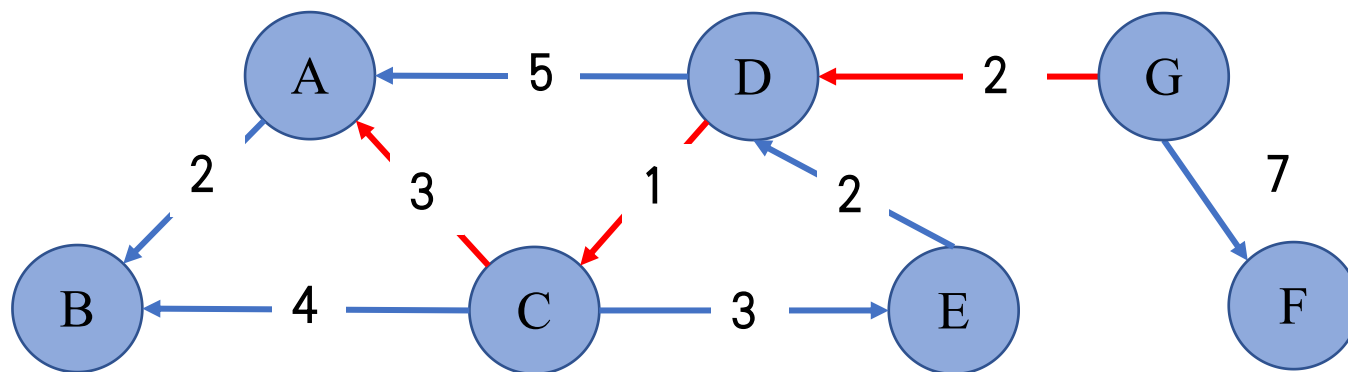
$$\text{路径长度} = c_1 + c_2 + \dots + c_m$$
$$\text{路径: } (v, v_1, v_2, \dots, u)$$

从源点到终点可能不止一条路径，把路径长度最短的那条路径称为最短路径。

最短路径问题



示例:



$$\delta(G, A) = W(p) = 2 + 1 + 3 = 6$$

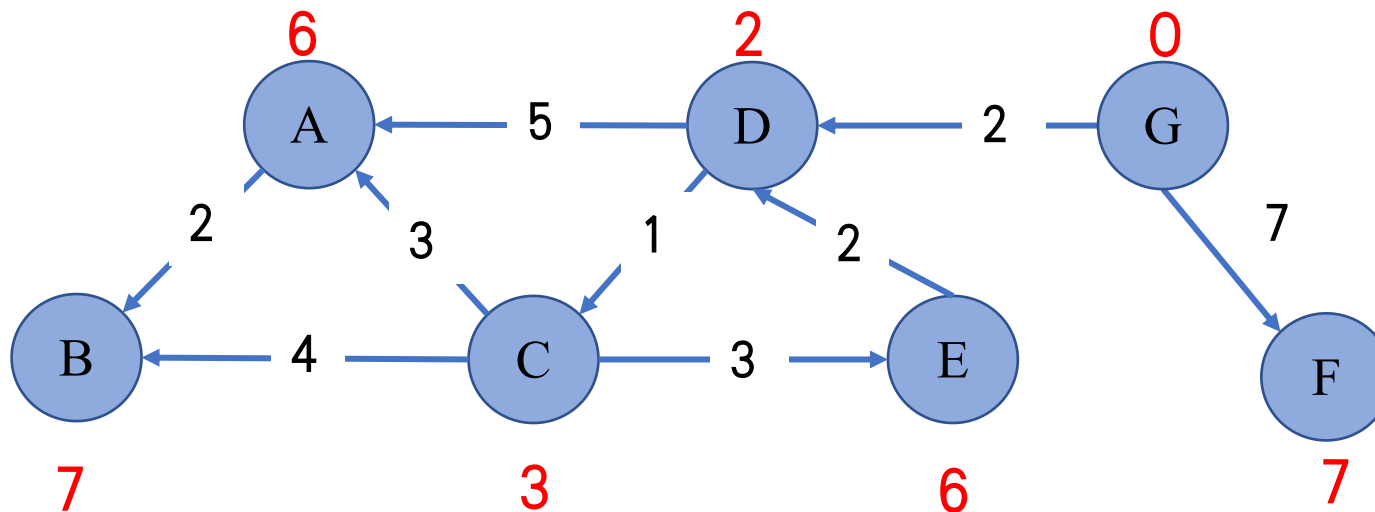
$$\delta(A, G) = \infty$$

最短路径问题



单源最短路径问题:

给定一个带权图 $G=(V, E, W)$ 和一个源点 s , 计算 s 到其他 V 中所有点的距离。

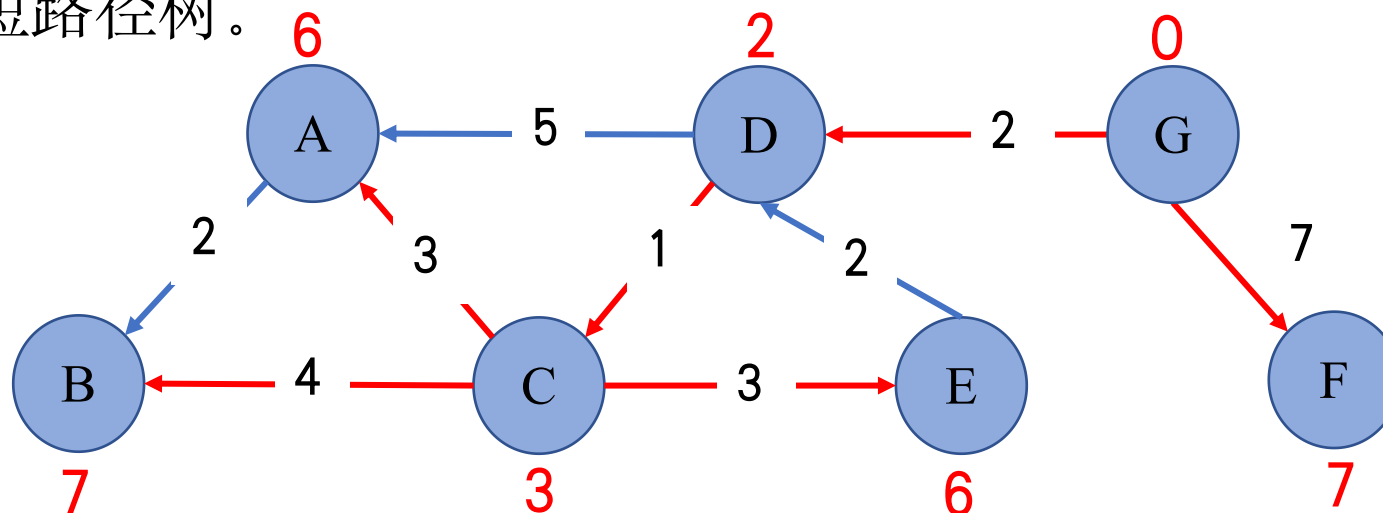


最短路径问题



最短路径树:

由源点s到所有点的最短路径合起来就形成一棵树，称为s的最短路径树。



为了得到最短路径树，计算任一点v在树上的父节点，记为 $p[v]$ 。

目录

第一节

最短路径问题

第二节

松弛算法

第三节

有向无环图上的
最短路径计算

第四节

Dijkstra算法

第五节

Bellman-Ford算法

第六节

差分约束和最
短路径

第六节

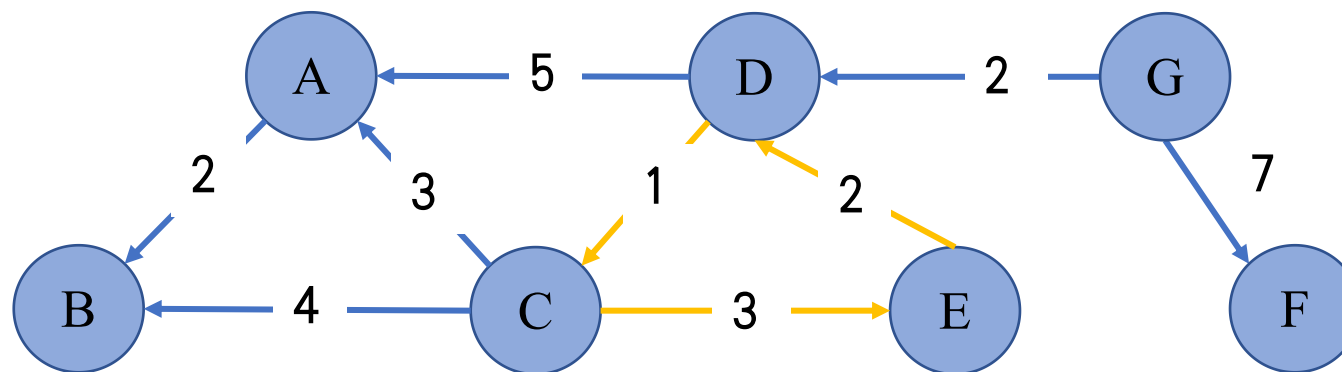
路径计算前沿

松弛算法



求解最短路径的蛮力法:

为了计算s到t的最短路径，枚举出s到t的所有路径并计算最小值, 如果有环，路径数量是无穷多。





求解最短路径的蛮力法:

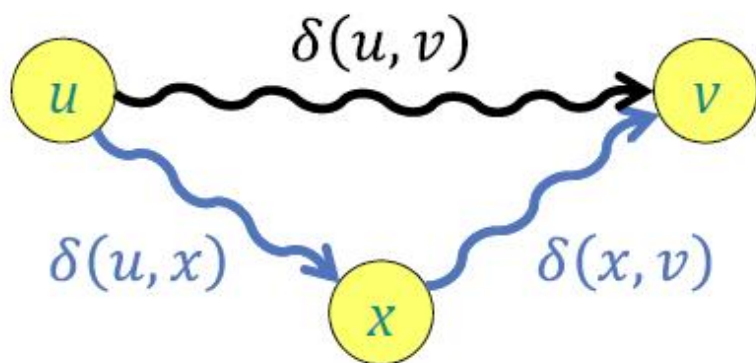
为了计算s到t的最短路径，枚举出s到t的所有简单路径并计算最小值，路径数量是指数级别的 $O(|V|!)$ 。

松弛算法



三角不等式:

对于图上任意三个点 u 、 v 和 x 而言, $\delta(u,v) \leq \delta(u,x) + \delta(x,v)$ 。



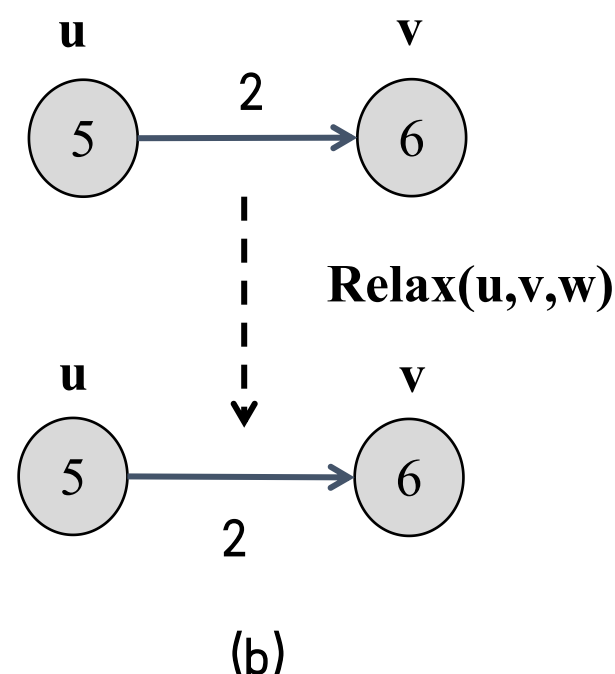
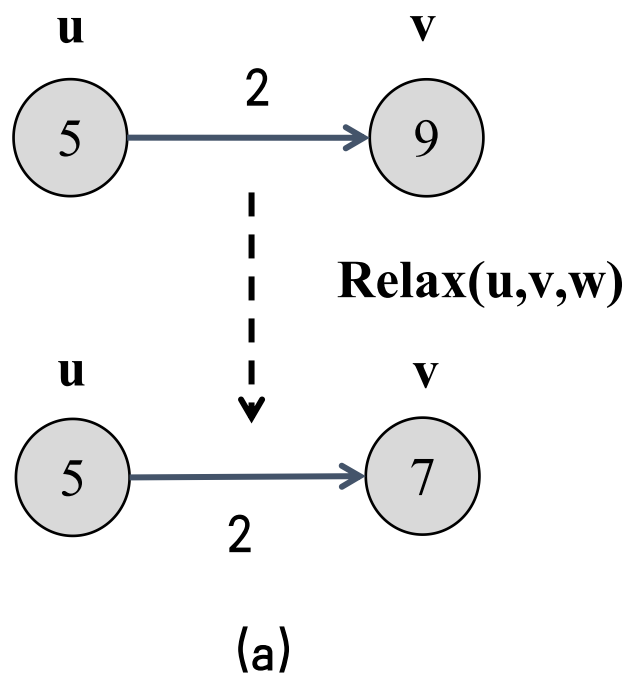
推论: 对于图上任意三个点 u 、 v 和 x 且存在边 (x,v) 而言,
 $\delta(u,v) \leq \delta(u,x) + w(x,v)$ 。

松弛算法



对每个点 v 而言，维护一个属性 $d[v]$ ，记录源点 s 到点 v 的距离上界，不断地降低对 $d[v]$ 的估计，直到 $d[v]$ 等于 $\delta(s,v)$ 。

- Relax(G, s)
 - 1 for v in V :
 - 2 $d[v] = \infty$;
 - 3 $p[v] = \text{NULL}$;
 - 4 $d[s] = 0$; $p[s] = s$;
 - 5 while some $e(u, v)$ has $d[v] > d[u] + w(u, v)$:
 - 6 pick such an $e(u, v)$
 - 7 $d[v] = d[u] + w(u, v)$;
 - 8 $p[v] = u$;





收敛性质:

对于某些节点 $u, v \in V$, 如果 $s \rightarrow u \rightarrow v$ 是图 G 中的一条最短路径, 并且对边 (u, v) 进行松弛前的任意时间有 $d[u] = \delta(s, u)$, 则在对边 (u, v) 松弛之后的所有时间 $d[v] = \delta(s, v)$

松弛算法



路径松弛性质:

给定带权图 $G=(V, E, W)$, 设从源点 s 到点 v_k 的最短路径为 $se_1v_1e_2 \dots e_kv_k$, 如果对边 $e_1, e_2, \dots, e_k$ 按次序进行松弛操作之后, $d[v_k]=\delta(s, v_k)$, 且该性质的成立与其他边的松弛操作以及次序无关。

证明: 数学归纳法, 对于路径边数进行归纳

基础步: 边数为0的最短路径就是源点, 初始化阶段就有 $d[s]=\delta(s, s)=0$;

归纳步: 边数为 n 的最短路径都成立, 边数为 $n+1$ 最短路径 $se_1v_1e_2 \dots e_nv_n e_{n+1}v_{n+1}$ 有两部分组成: 边数为 n 的最短路径和只有一条边 e_{n+1} 的一条最短路径。

边数为 n 的最短路径按归纳假设有 $d[v_n]=\delta(s, v_n)$, 于是按照松弛操作有 $d[v_{n+1}]=d[v_n]+w(e_{n+1})=\delta(s, v_{n+1})$

松弛算法



松弛算法正确性（上界性质）：

定理：在松弛算法中，对于 V 中任意的 v ， $d[v]$ 一直大于等于 $\delta(s, v)$ （最短路径），且一旦 $d[v]$ 到达 $\delta(s, v)$ 之后将不再发生变化。

证明：数学归纳法，针对调用松弛操作的次数

基础步：当松弛操作次数为0的时候，即初始化阶段， $d[s] = 0 \geq \delta(s, s) = 0$ ，而其它点 v 的 $d[v]$ 都是 ∞ 都是大于等于 $\delta(s, v)$ 。

归纳步：当松弛操作次数小于等于 n 时候，设第 $n+1$ 次松弛操作加入了边 (u, v) 。

由于 $d[u]$ 是在前 n 中操作中得到的值，所以 $\delta(s, u) \leq d[u]$ ，于是，由三角不等式， $\delta(s, v) \leq \delta(s, u) + w(u, v) \leq d[u] + w(u, v)$ ，而 $d[u] + w(u, v) = d[v]$ 就是第 $n+1$ 次松弛操作的结果。

松弛算法



前驱子图性质:

对于所有的节点 $v \in V$, 一旦 $d[v] = \delta(s, v)$, 则前驱子图是一颗根节点为 s 的最短路径树。

目录

第一节

最短路径问题

第二节

松弛算法

第三节

有向无环图上的
最短路径计算

第四节

Dijkstra算法

第五节

Bellman-Ford算法

第六节

差分约束和最
短路径

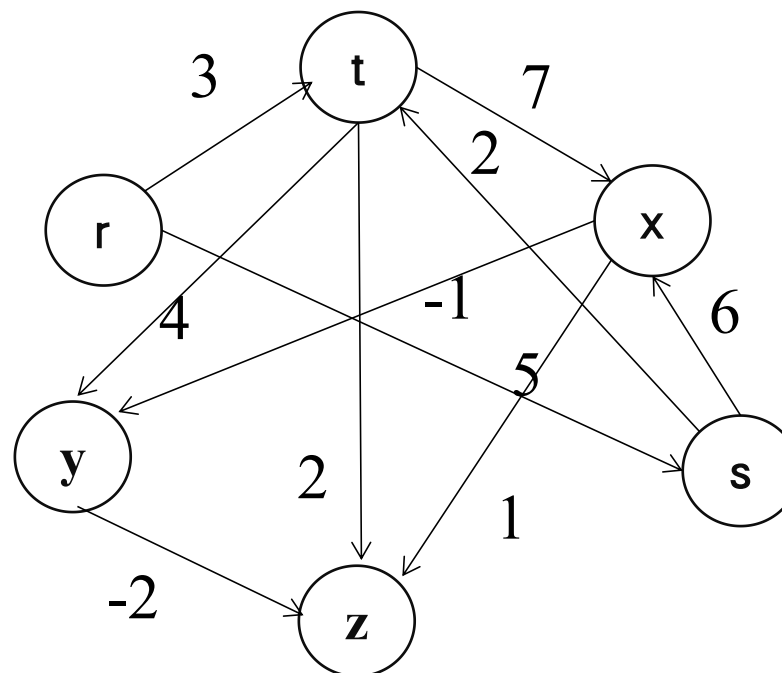
第六节

路径计算前沿

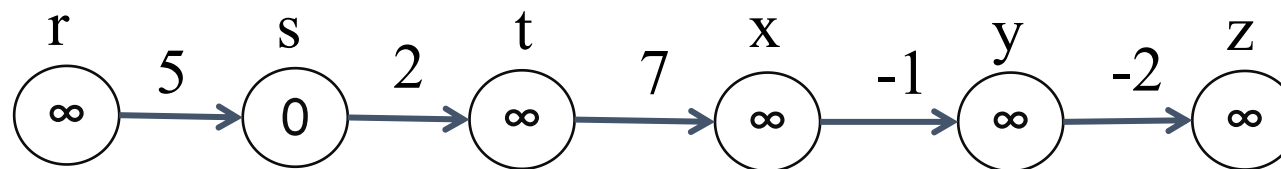
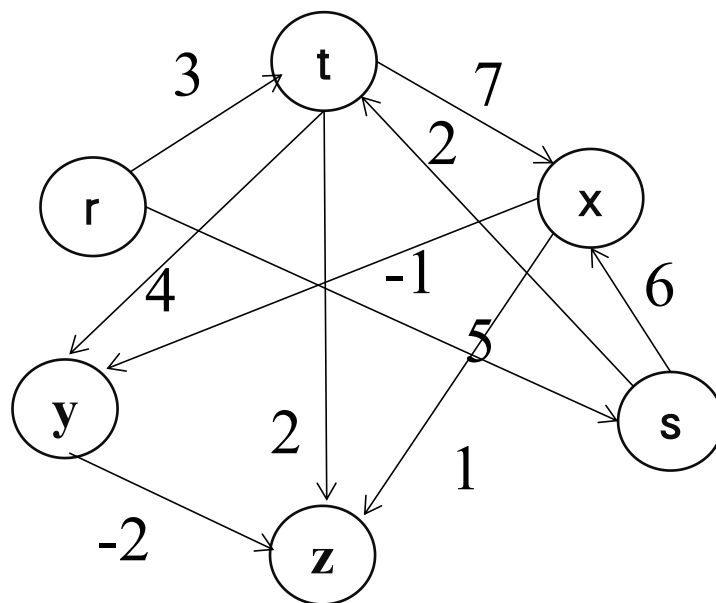
有向无环图上的最短路径计算



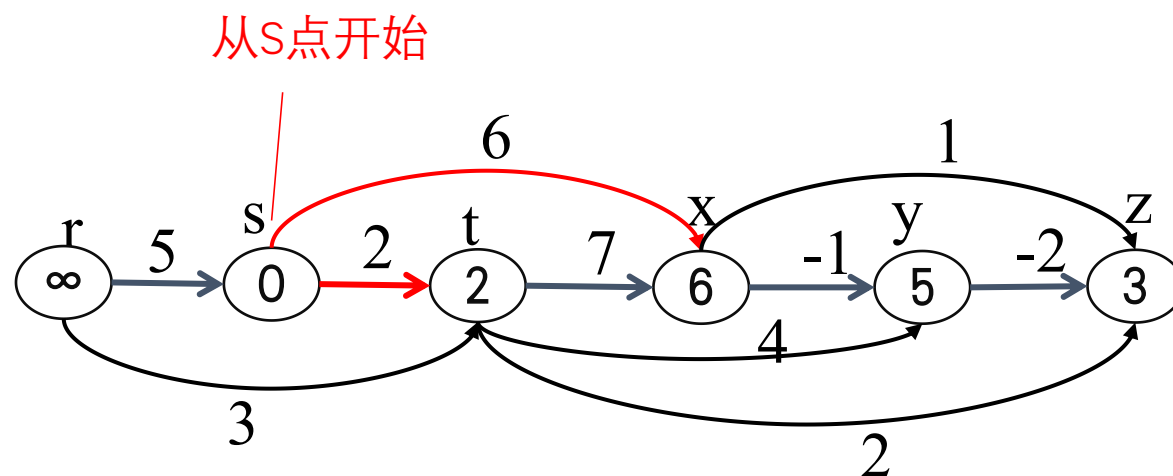
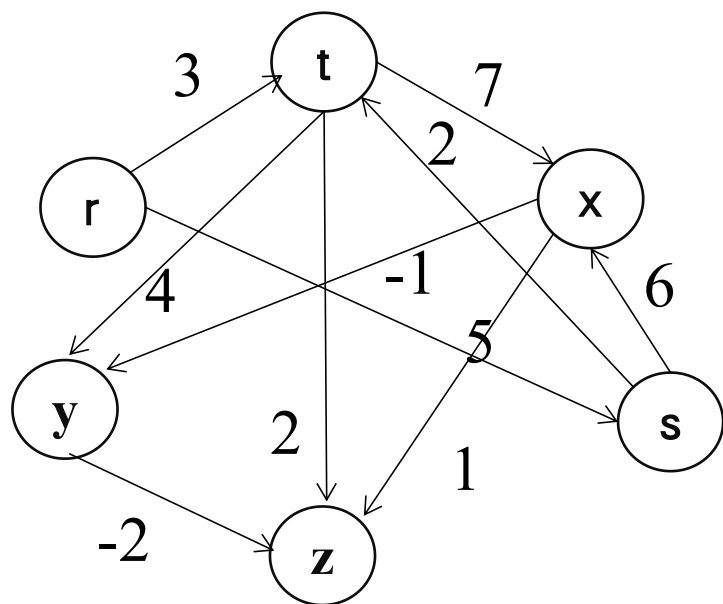
- ◆ 首先，对有向无环图中所有点进行拓扑排序；
- ◆ 然后，按照拓扑排序的顺序对所有边进行操作松弛。
- ◆ 时间复杂度 $O(|V|+|E|)$ 。



有向无环图上的最短路径计算



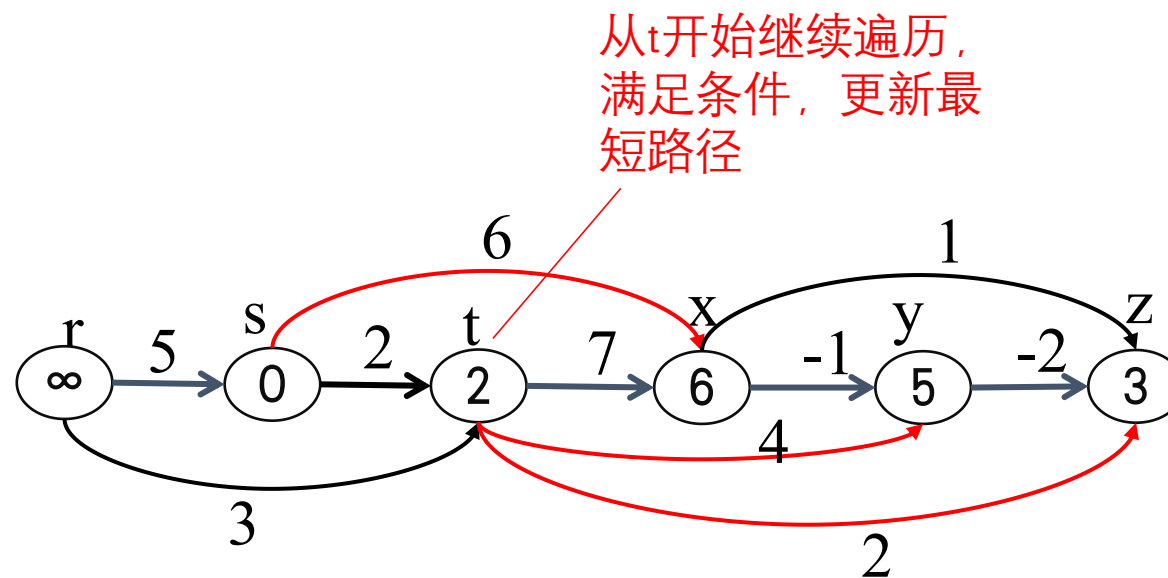
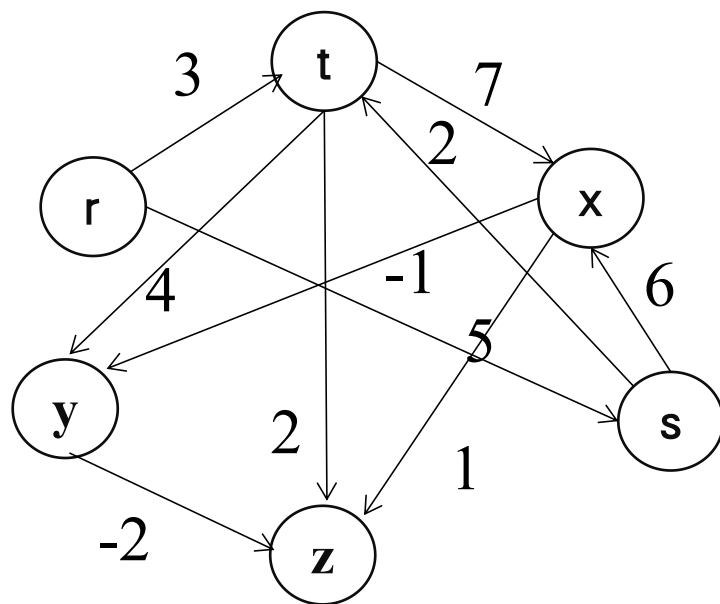
有向无环图上的最短路径计算



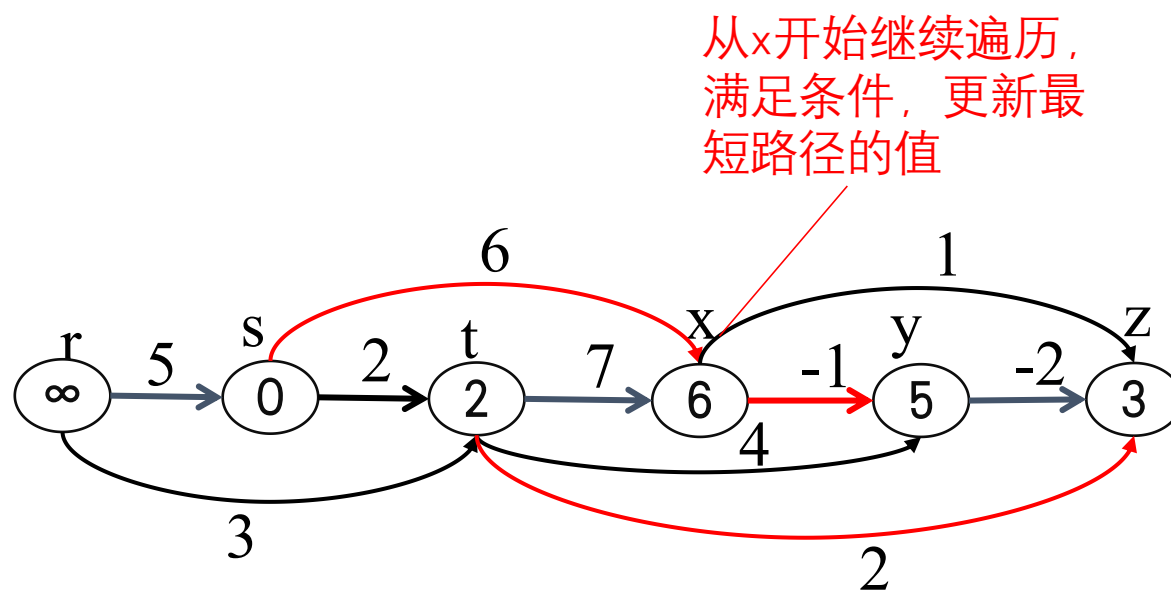
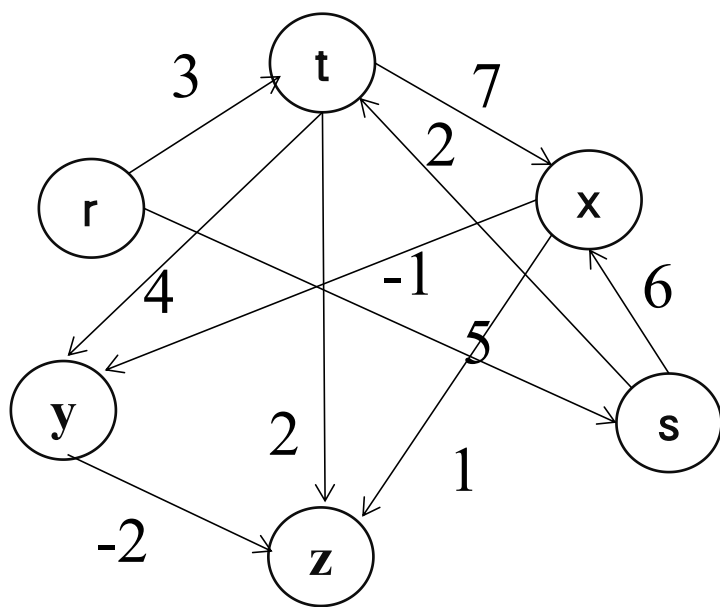
- 1 if($d[v] > d[u] + w(u, v)$)
- 2 then $d[v] \leftarrow d[u] + w(u, v)$
- 3 $\pi[v] \leftarrow u$ /*解释[松弛操作](#)*/

s到t的最短路径长度为2,因此修改 $d[t]=2$,s到x的目前最短路径为6,因此修改 $d[x]=6$,一次松弛操作完成,结果如上图所示。

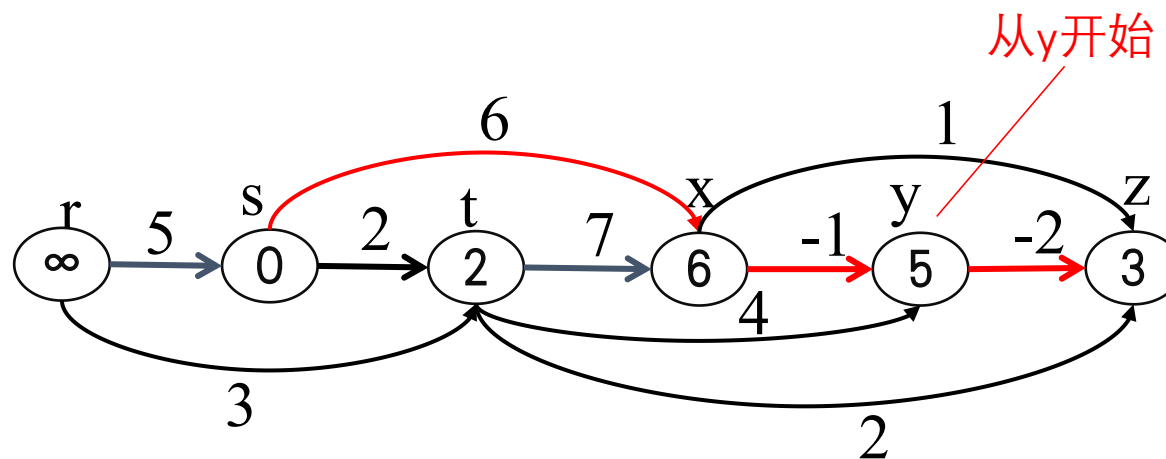
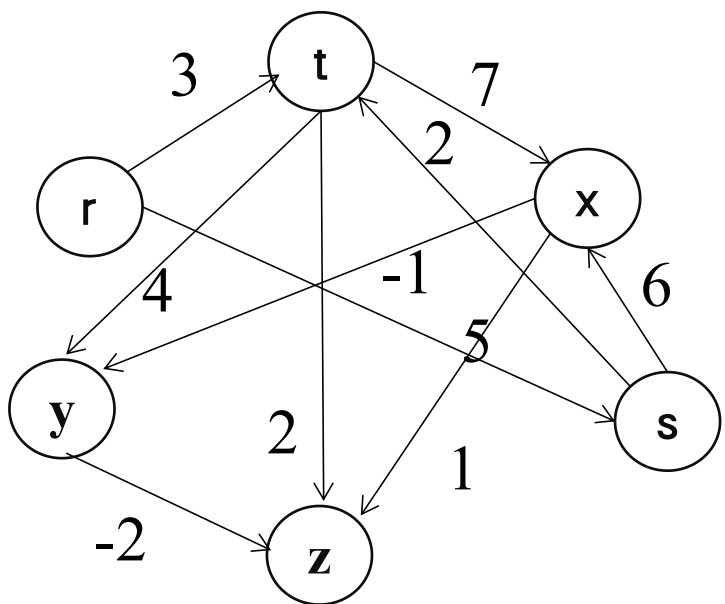
有向无环图上的最短路径计算



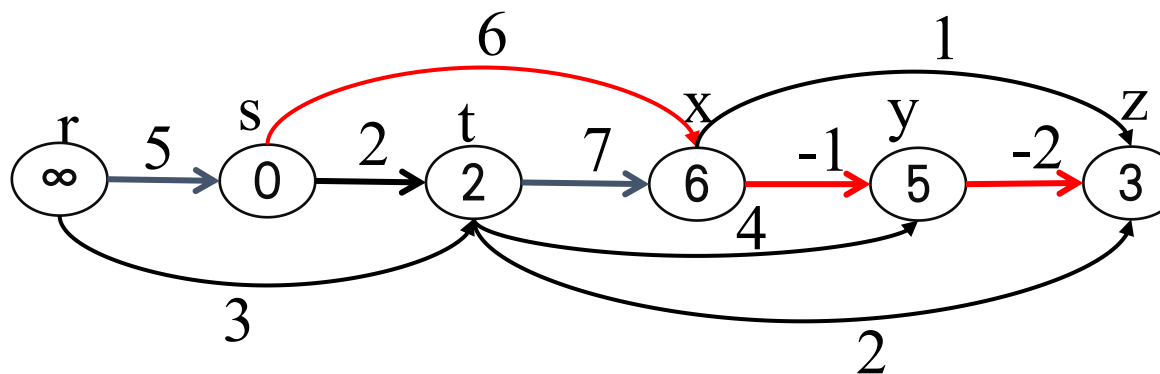
有向无环图上的最短路径计算



有向无环图上的最短路径计算



最终结果



目录

第一节

最短路径问题

第二节

松弛算法

第三节

有向无环图上的
最短路径计算

第四节

Dijkstra算法

第五节

Bellman-Ford算法

第六节

差分约束和最
短路径

第六节

路径计算前沿

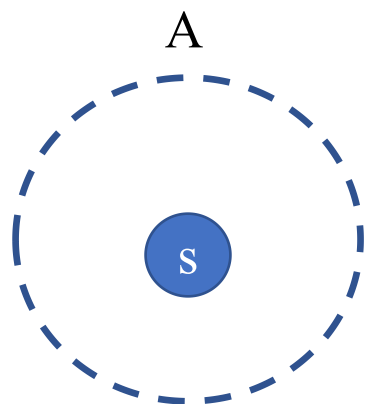
Dijkstra算法



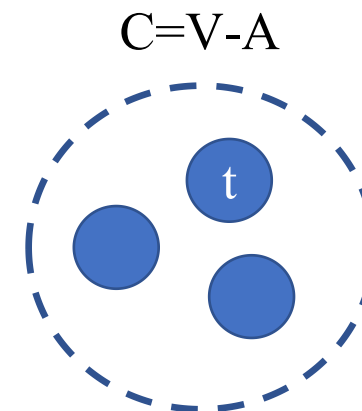
狄克斯特拉 (Dijkstra) 求解思路:

设 $G=(V, E)$ 是一个带权有向图, 把图中顶点集合 V 分成两组:

- 第1组为其余未求出最短路径的顶点集合 (用 C 表示)
- 第2组为已求出最短路径的顶点集合 (用 A 表示, 初始时 A 为空, 以后每一轮将 $d[t]$ 最小的点 t 拿入 A 。第一轮拿的是 s , 并**对 s 的出边进行松弛**)



每一步求出 s 到 C 中一个顶点 t 的最短路径, 并将 t 移动到 A 中。直到 C 为空。

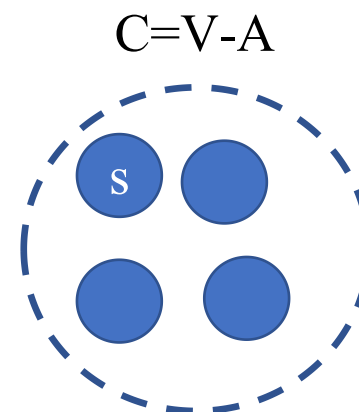
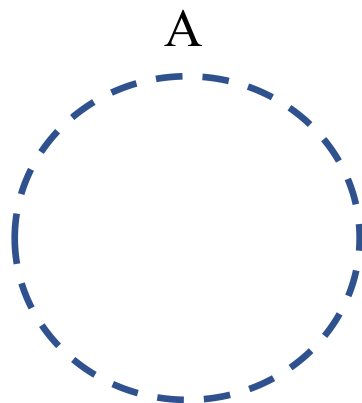


Dijkstra算法



Dijkstra算法的过程:

(1) 初始化: A 为空, s 的最短路径为0, 仅仅 $d[s]$ 被初始化为0, 其余任一点 v 都初始化为 $d[v]=\infty$

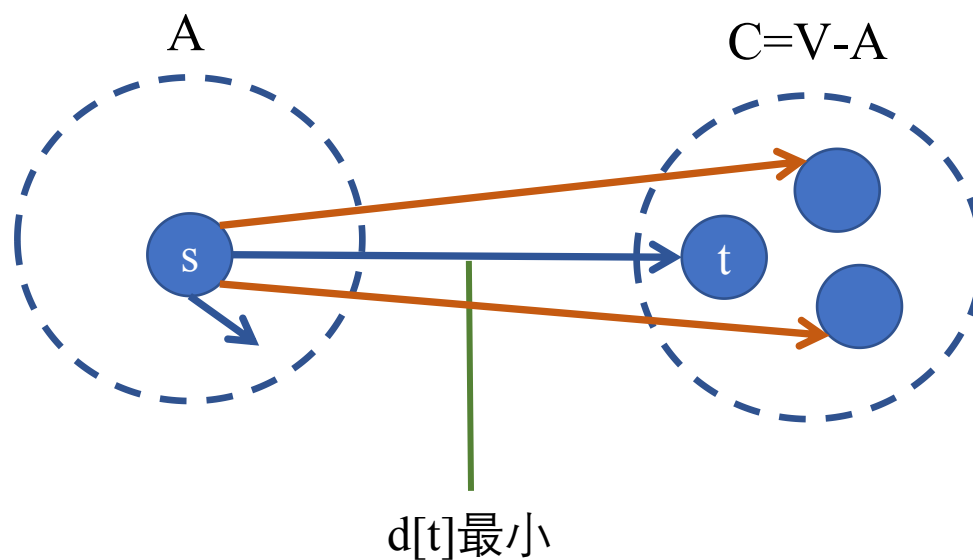


Dijkstra算法



Dijkstra算法的过程:

(2) 从C中选取一个 $d[t]$ 距离s最小的顶点t, 把t加入A中 (该选定的距离就是 $s \Rightarrow t$ 的最短路径长度)。其中第一轮就是选取s本身。

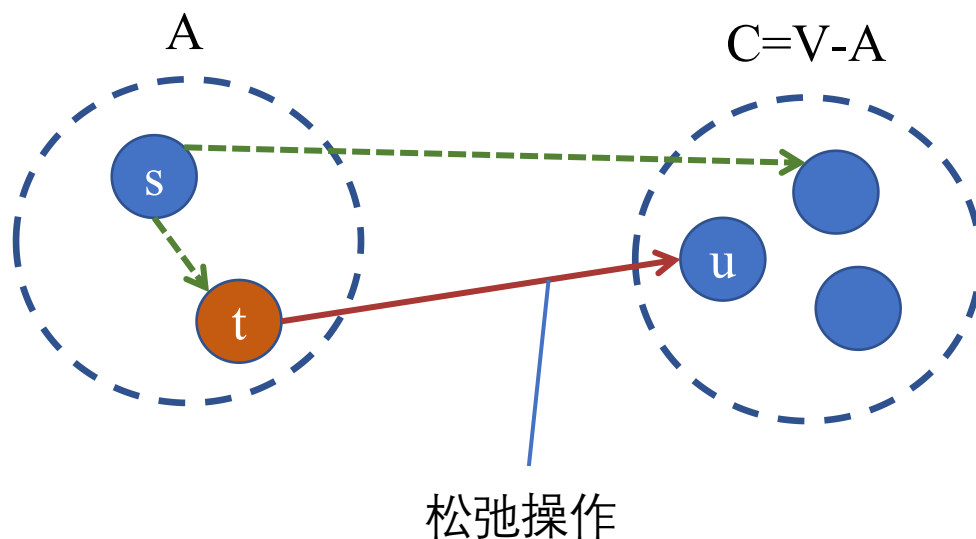


Dijkstra算法



Dijkstra算法的过程:

(3) 对t的相邻边进行松弛



Dijkstra算法



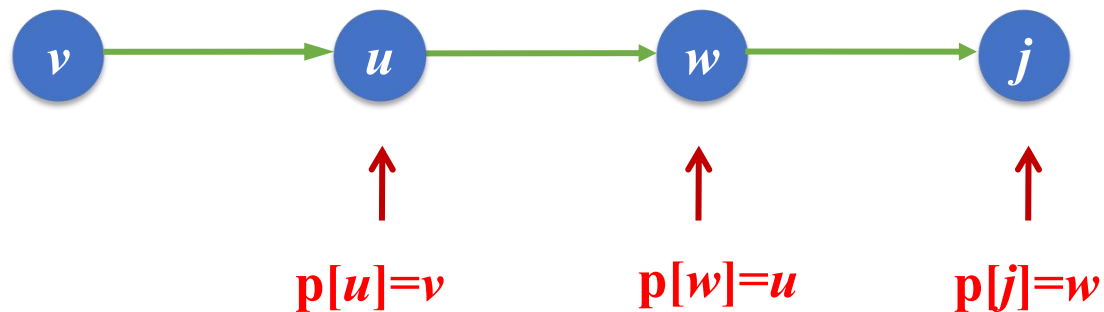
Dijkstra算法的过程:

(4) 重复步骤 (2) 和 (3) 直到所有顶点都包含在A中。

Dijkstra算法



$v \Rightarrow j$ 的最短路径:



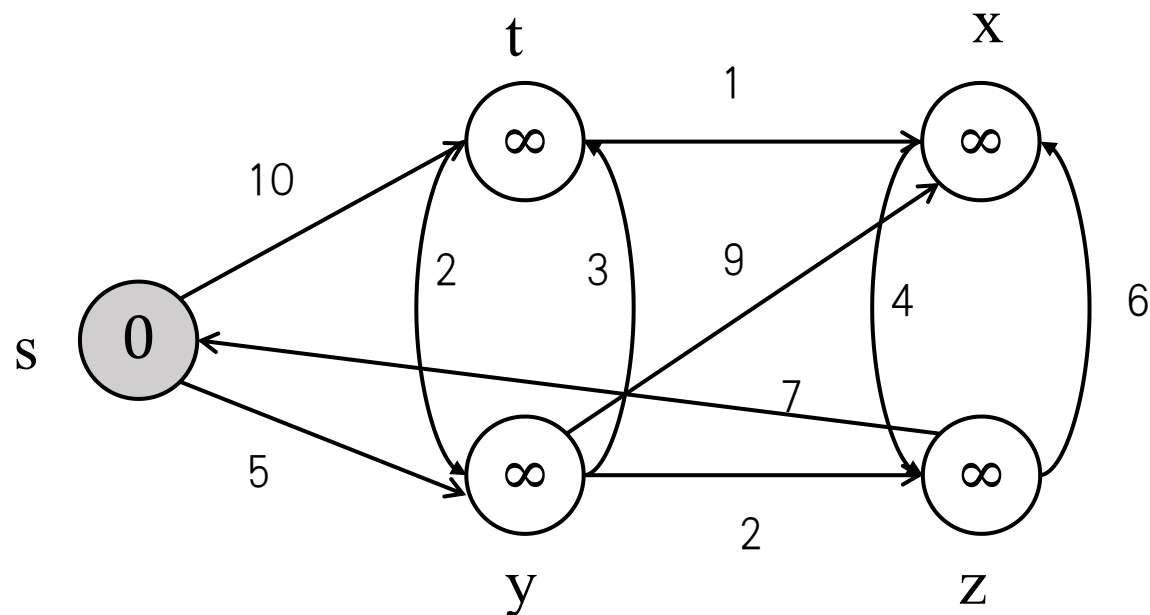
从 $p[j]$ 推出的逆路径: j, w, u, v

对应的最短路径为: $v \rightarrow u \rightarrow w \rightarrow j$

Dijkstra算法



Dijkstra算法 示例演示

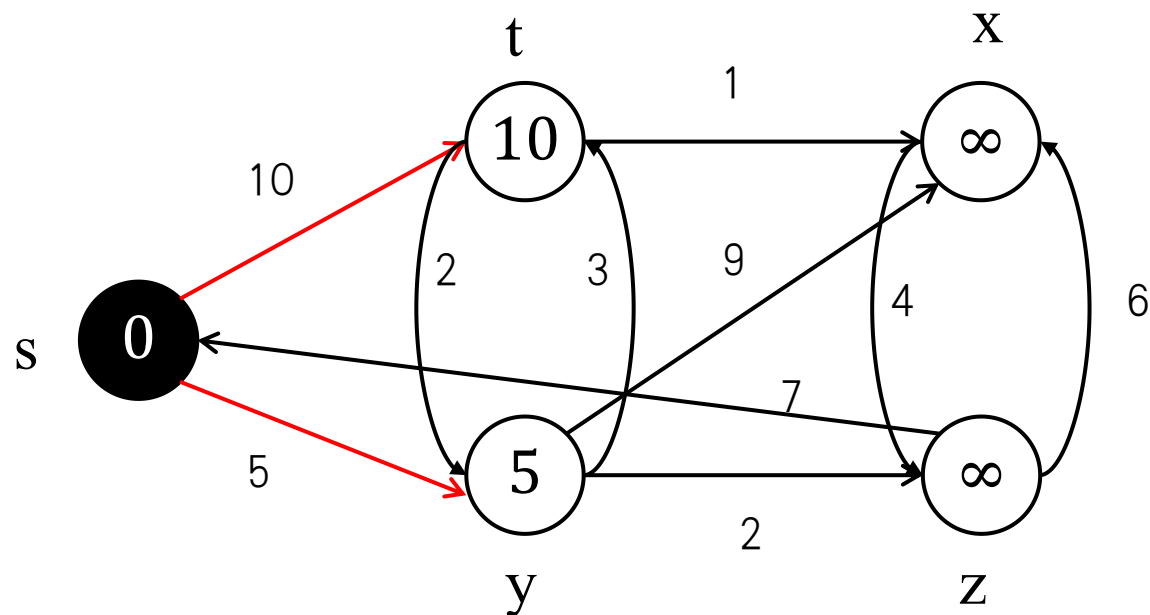


黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



Dijkstra算法 示例演示

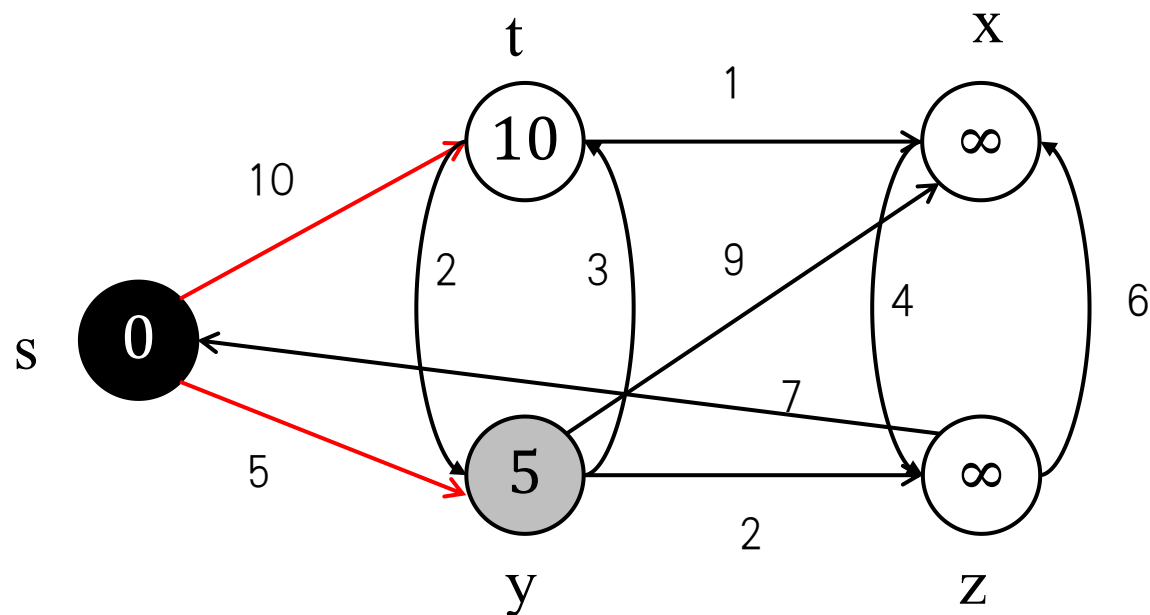


黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



Dijkstra算法 示例演示

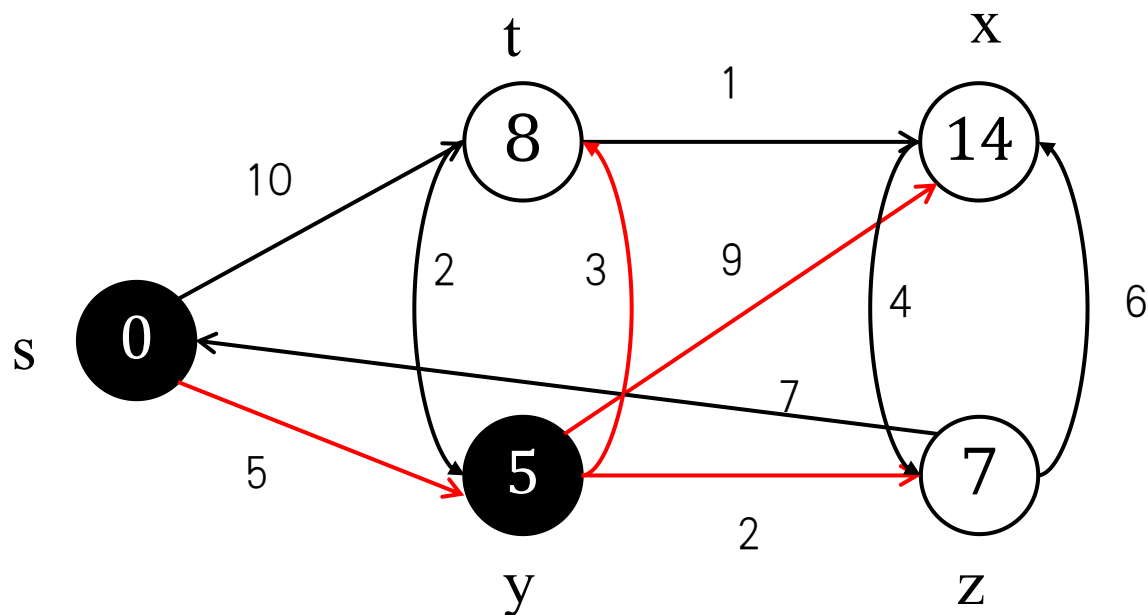


黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



Dijkstra算法 示例演示

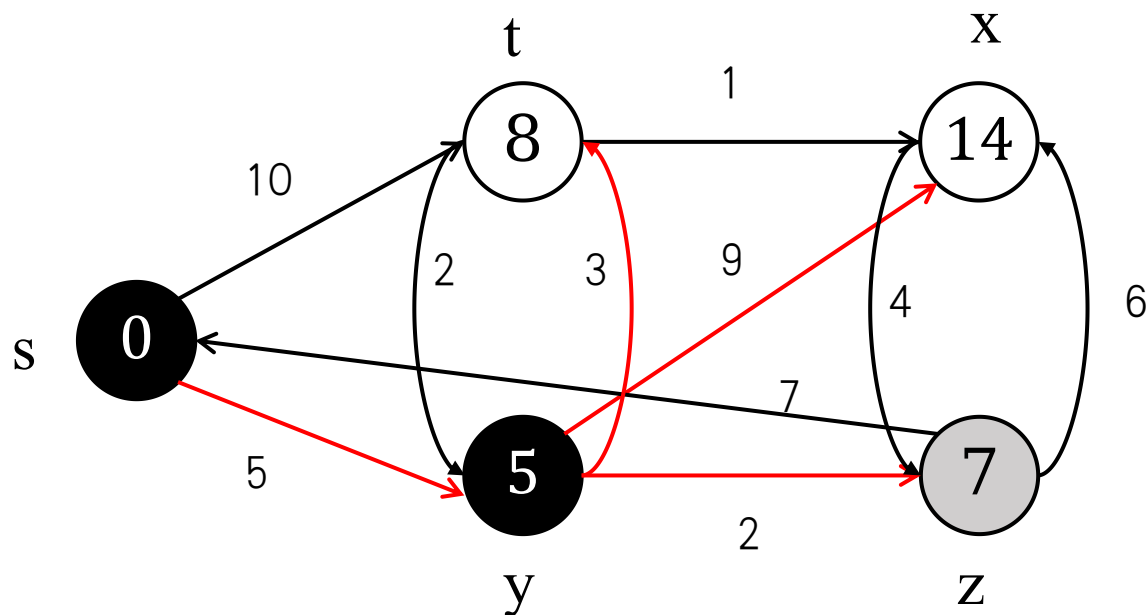


黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



Dijkstra算法 示例演示

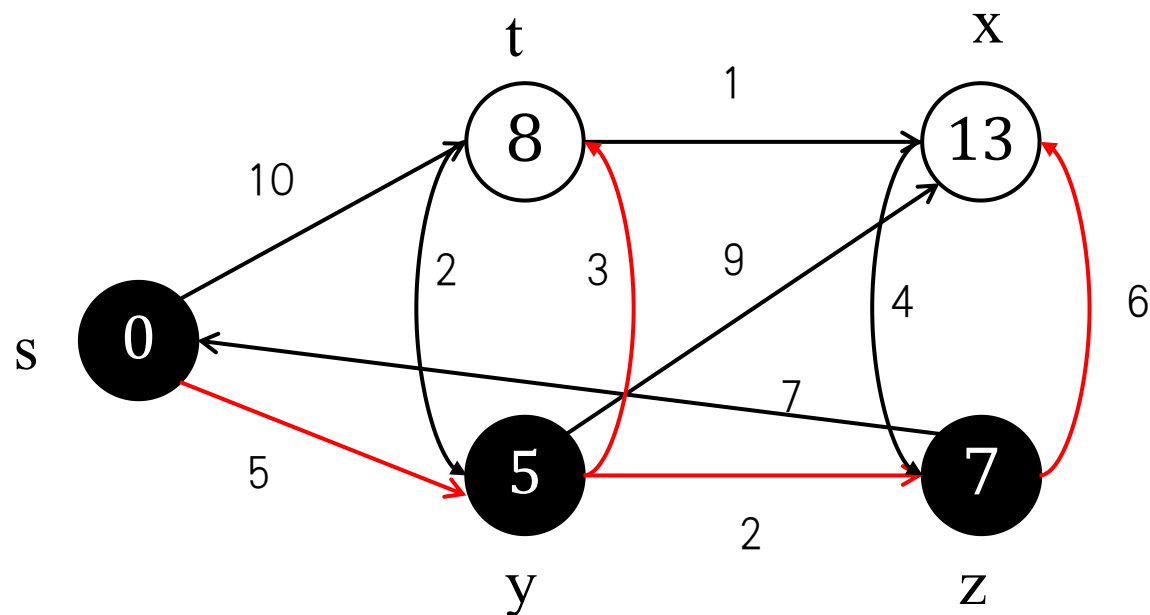


黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



Dijkstra算法 示例演示

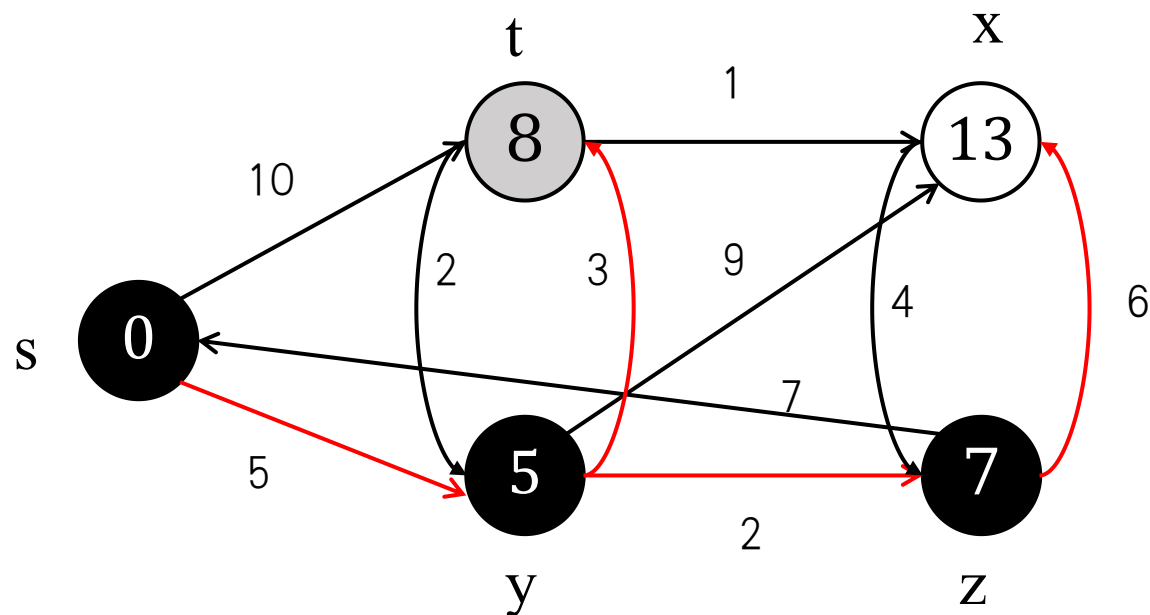


黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



Dijkstra算法 示例演示

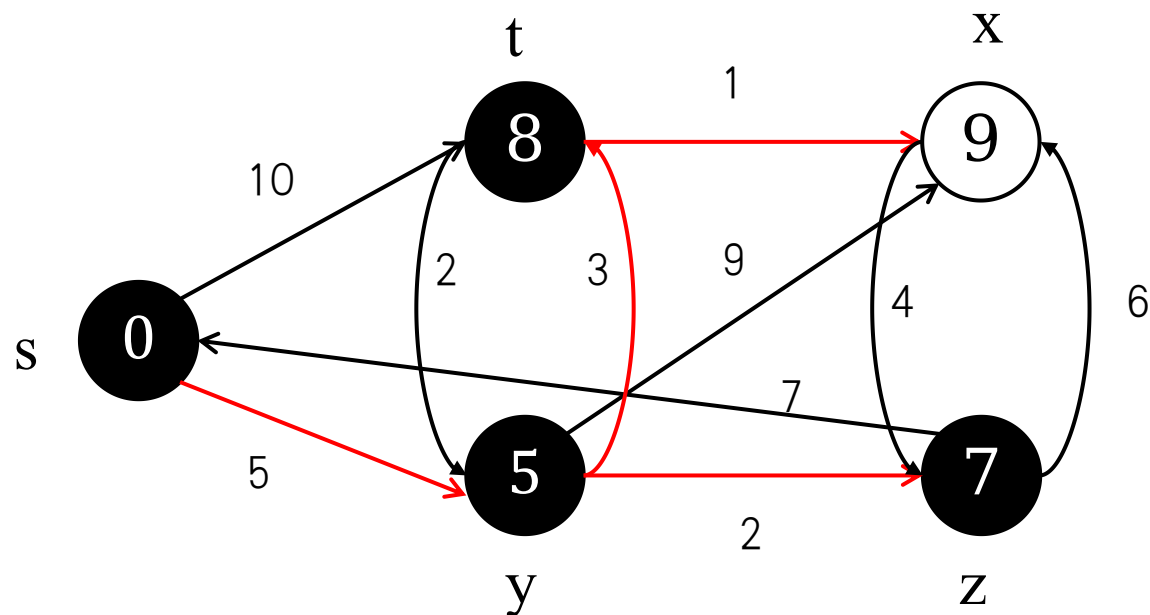


黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



Dijkstra算法 示例演示

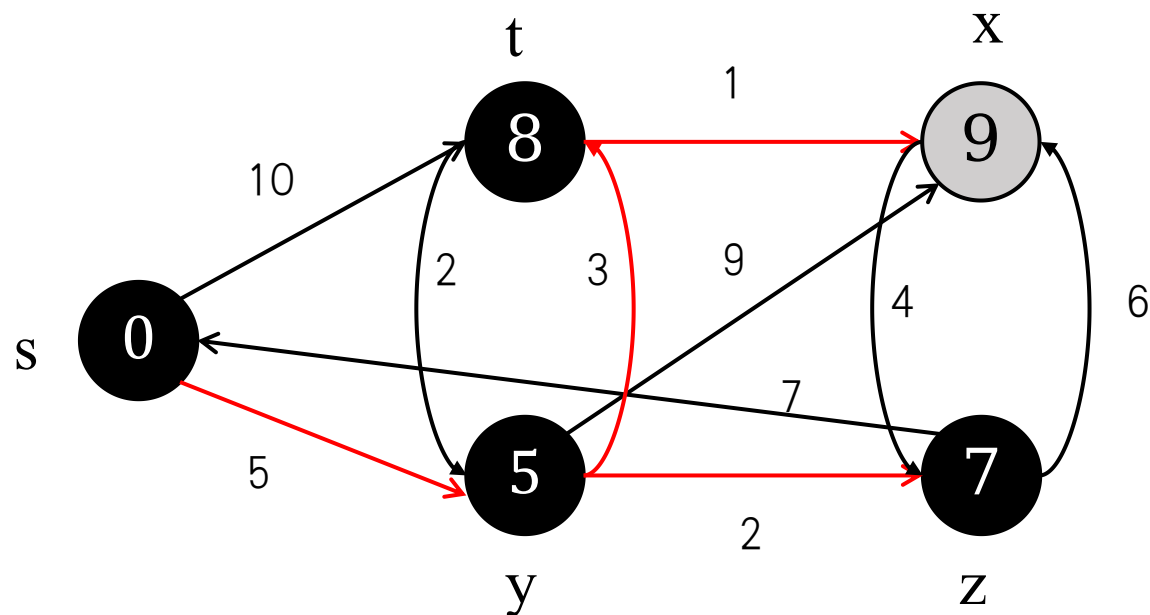


黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



Dijkstra算法 示例演示

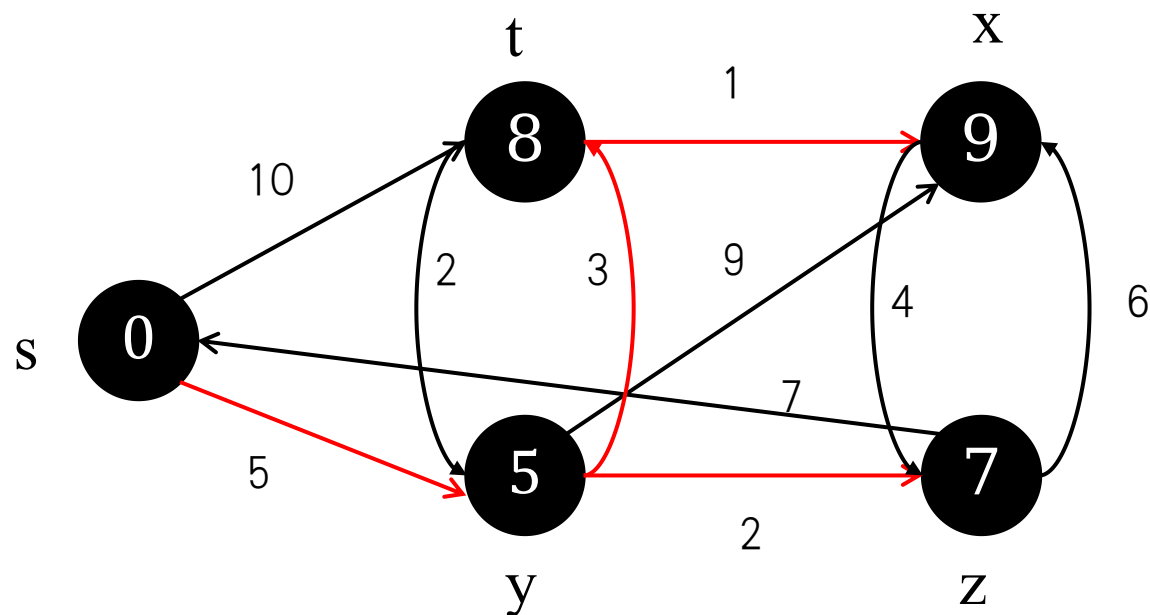


黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



Dijkstra算法 示例演示



黑色点表示已经在A中的点；灰色和白色点是在C中点，其中灰色是C中d值最小的点

Dijkstra算法



伪代码:

Dijkstra(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 $A = \emptyset$

3 $Q = G.V$

4 **while** $Q \neq \emptyset$

5 $u = \text{EXTRACT-MIN}(Q)$

6 $A = A \cup \{u\}$

7 **for** each vertex $v \in G.\text{Adj}[u]$

8 RELAX(u, v, w)

Dijkstra算法



算法正确性:

给定带权图 $G=(V, E, W)$, 算法终止的时候 $d[v]=\delta(s, v)$

证明: 反证法

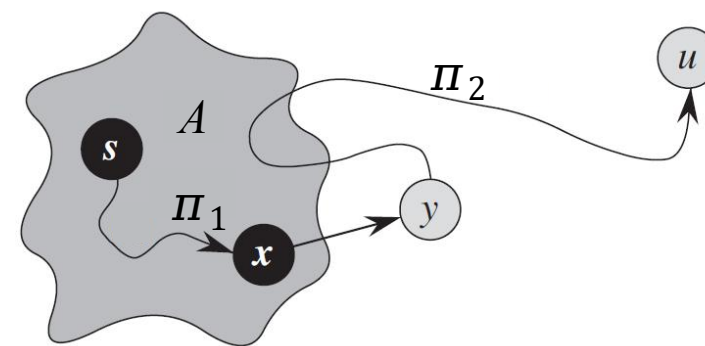
假设 u 是第一个出优先队列时 $d[u] \neq \delta(s, u)$ 的点,

必然存在一条最短路径 π 是 s 到 u ,

x 是 π 中 u 出优先队列时最后一个进 S 的点,

y 是 x 在 π 中的后继, 显然 $d[y]$ 小于等于 $d[u]$,

所以 y 肯定要在 u 之前出队列, 与假设矛盾



Dijkstra算法



时间复杂度:

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(E + V \lg V)$ worst case

Dijkstra算法



思考题:

Dijkstra算法可以用于带权无向图求最短路径吗?

思考题:

Dijkstra算法是否一定可以停止吗?

思考题:

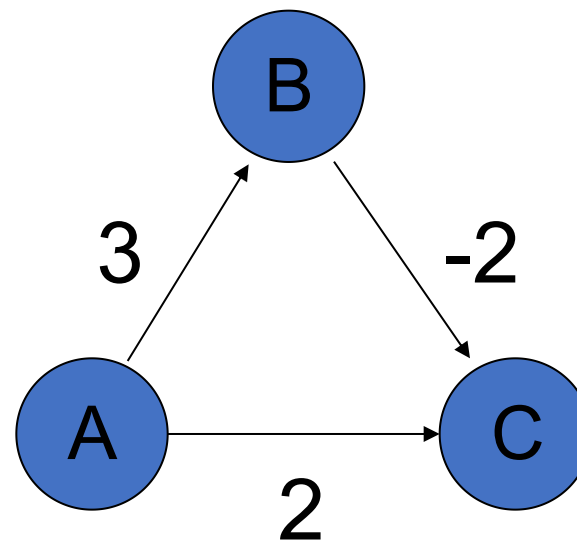
Dijkstra算法为什么不适合负权值的情况?

Dijkstra算法的局限性



Dijkstra算法的局限性:

- 如果边权为负值，Dijkstra算法还正确吗？
- 求解右图A 至其他点的最短距离
- 算法步骤:
 - 1) 标记点A
 - 2) $d[C]=2$ 最小，标记点C
 - 3) $d[B]=3$ 最小，标记点B结束
- 但是 $\delta(A, C) = 1$



目录

第一节

最短路径问题

第二节

松弛算法

第三节

有向无环图上的
最短路径计算

第四节

Dijkstra算法

第五节

Bellman-Ford算法

第六节

差分约束和最
短路径

第六节

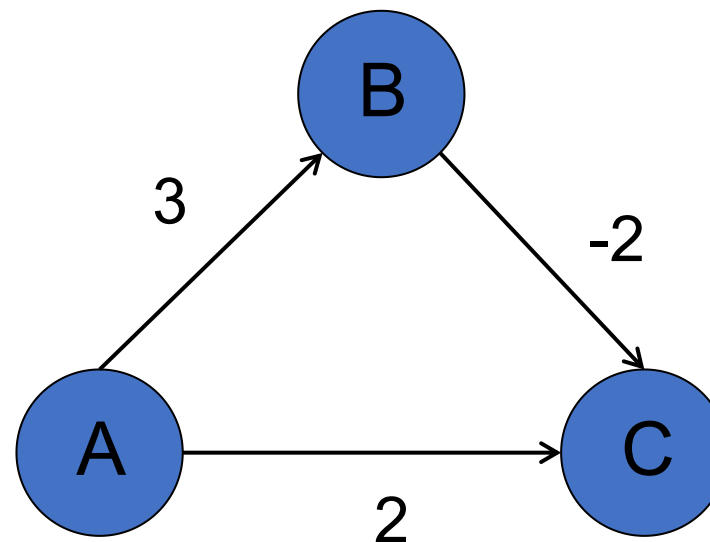
路径计算前沿

Bellman-Ford算法



Dijkstra算法的局限性:

- 如果边权为负值, Dijkstra算法还正确吗?
- 求解右图A 至其他点的最短距离
- 算法步骤:
 - 1) 标记点A
 - 2) $d[C]=2$ 最小, 标记点C
 - 3) $d[B]=3$ 最小, 标记点B结束
- 但是 $\delta(A, C) = 1$

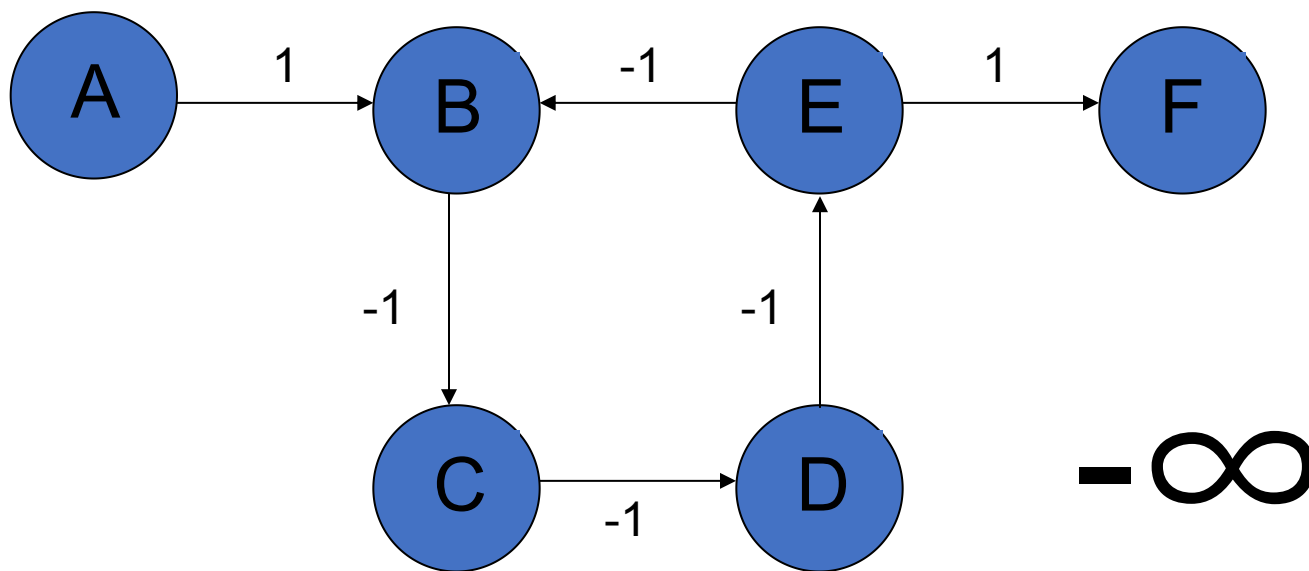


Bellman-Ford算法



Dijkstra算法的局限性:

下图中，A至F的最短路径长度是多少？

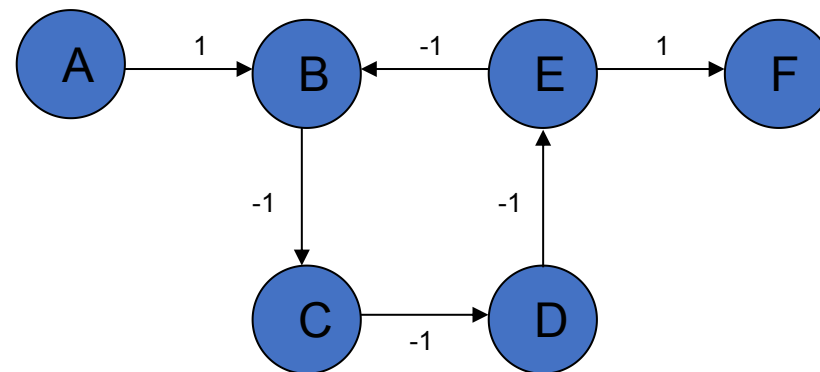


Bellman-Ford算法



Dijkstra算法的局限性:

- 如果利用Dijkstra算法求解，结果为……
- 标记点A， $d[B]=1$ ，标记点B
- $d[C]=0$ ，标记点C
- $d[D]=-1$ ，标记点D
- $d[E]=-2$ ，标记点E
- $d[F]=-1$ ，标记点F
- 所求得的距离并不是最短的



Bellman-Ford算法



Dijkstra算法的局限性：错误结果的原因

- Dijkstra的缺陷就在于它不能处理负权：Dijkstra对于标记过的点就不再更新了，所以即使有负权导致最短距离的改变也不会重新计算已经计算过的结果。
- 我们需要新的算法——Bellman-Ford

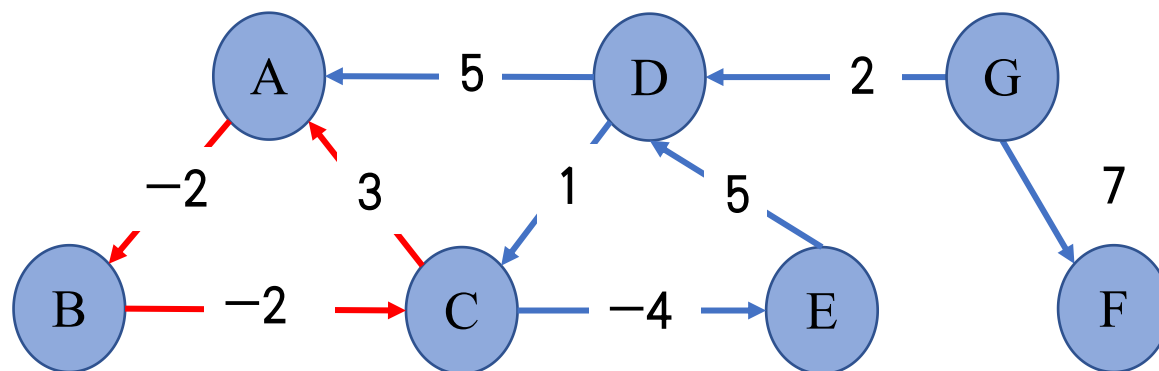
Bellman-Ford算法



负权环路:

给定一条路径 $p = v_0 e_1 v_1 e_2 \dots e_l v_l$, 如果若 v_0 和点 v_l 是同一个点, 则 p 被称为一条环路。

如果 p 中各条边权重之和 $\sum_{k=1}^{k=l} w(e_k)$ 是负数, 那么 p 称为负权环路。

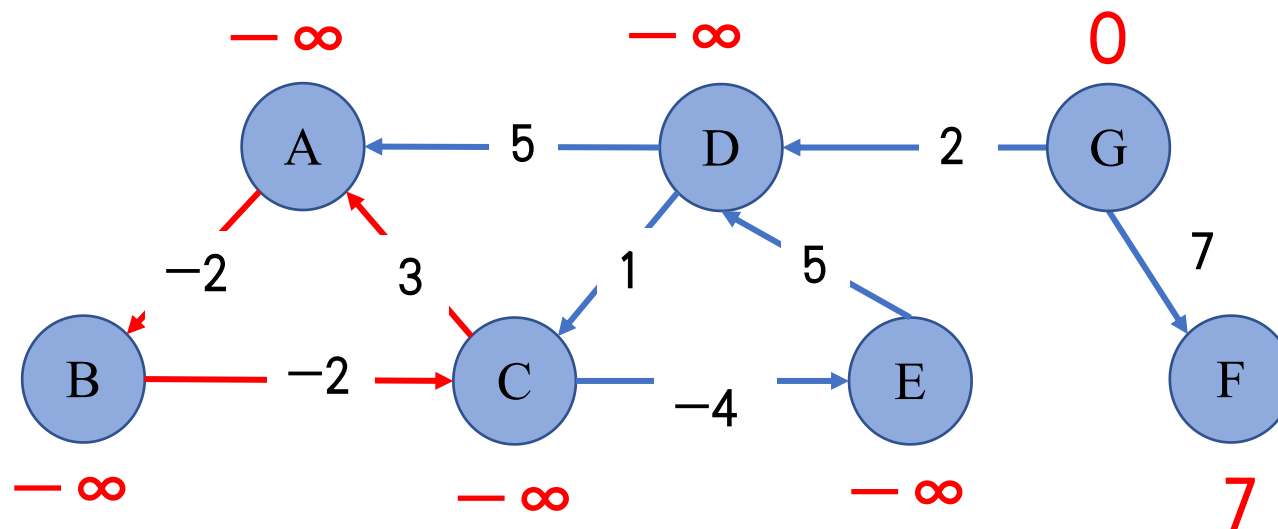


Bellman-Ford算法



基于负权环路的距离定义：

当图中有负权环路的时候，部分点的最短路径就未定义了，记为 $-\infty$



Bellman-Ford算法



算法伪代码:

```
1 Bellman-Ford(G,w,s)
2   for v in V:
3     d[v]=  $\infty$ ; p[v]=NULL;
4     d[s]=0;
5   for i from 1 to |V|-1 :
6     for each e (u, v)  $\in$  E:
7       Relax(u,v,w)
8   for each e (u, v)  $\in$  E:
9     if d[v] > d[u] + w(u, v)
10       return FALSE
11   return True
```

Bellman – Ford算法可以大致分为三个部分:

第一，初始化所有点。每一个点保存一个值，表示从原点到达这个点的距离，将原点的值设为0，其它的点的值设为无穷大（表示不可达）。

第二，进行循环，循环下标为从1到 $n-1$ （ n 等于图中点的个数）。在循环内部，遍历所有的边，进行松弛计算。

第三，遍历途中所有的边（ $e(u, v)$ ），判断是否存在这样情况：

$$d[v] > d[u] + w(u, v)$$

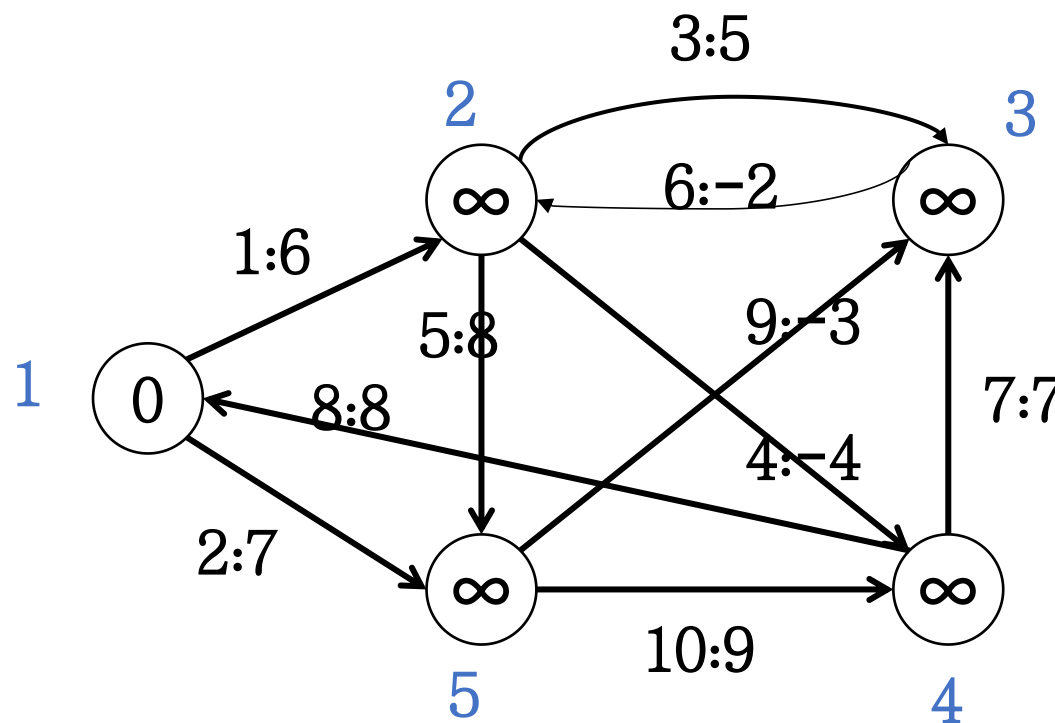
则返回false，表示途中存在从源点可达的权为负的回路。

Bellman-Ford算法

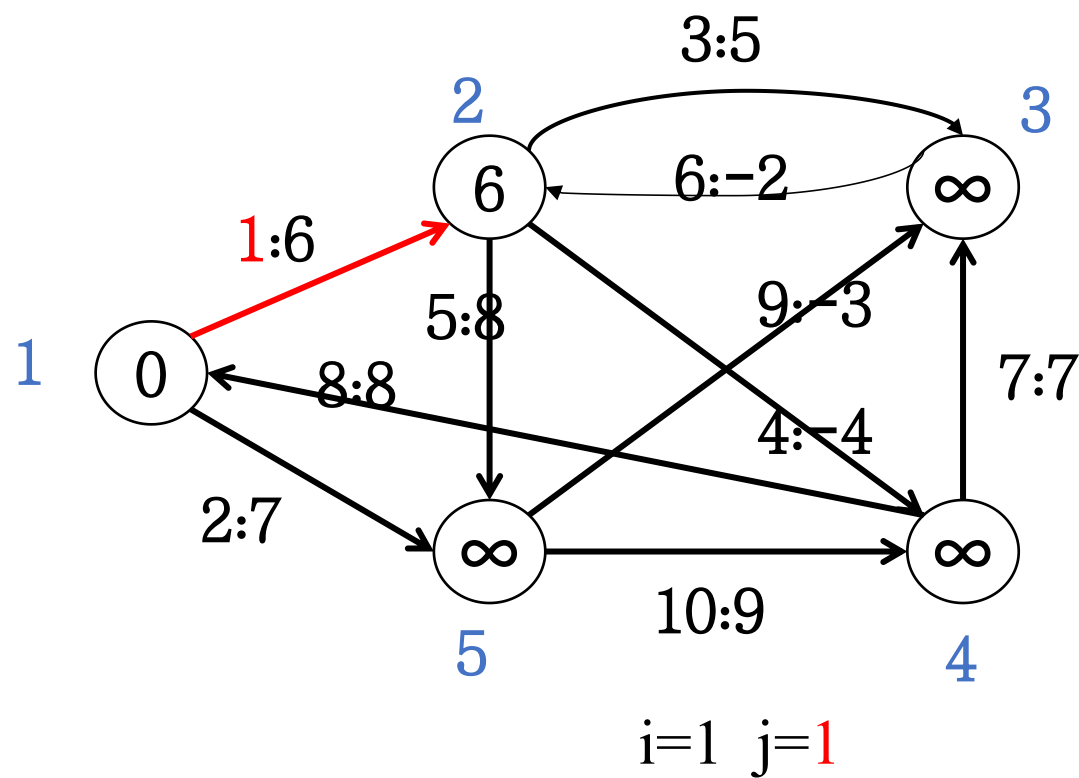


算法伪代码:

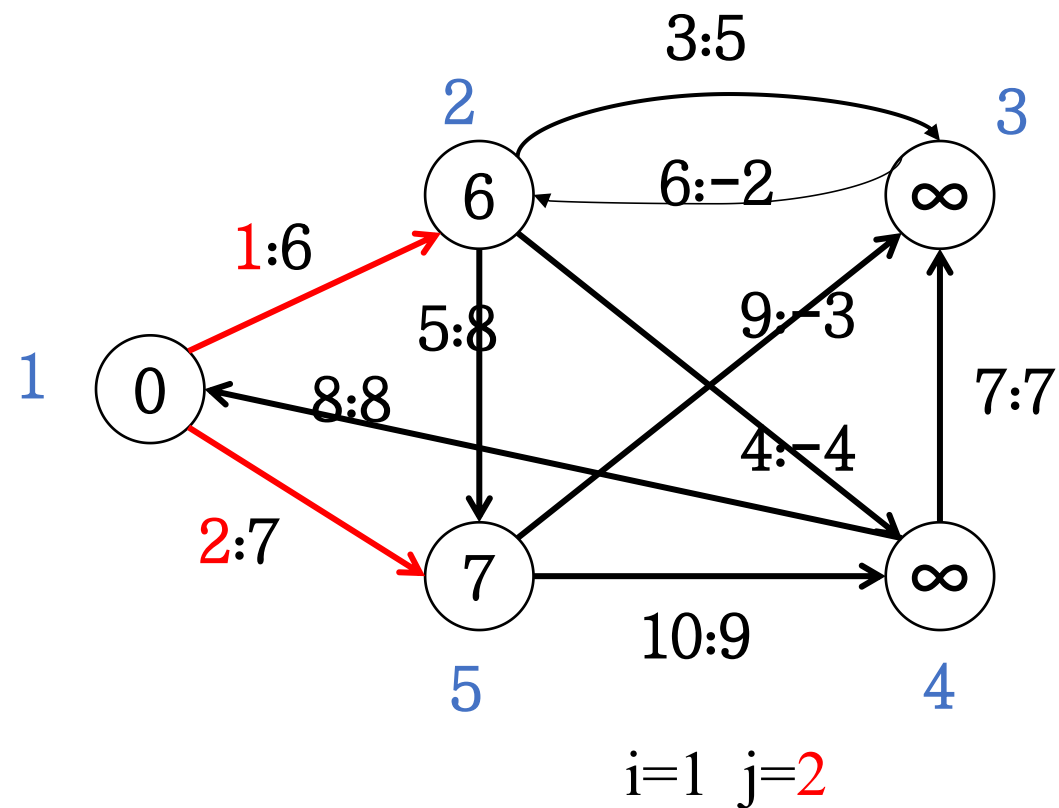
```
1 Bellman-Ford(G,w,s)
2   for v in V:
3     d[v]=  $\infty$ ; p[v]=NULL;
4     d[s]=0;
5   for i from 1 to |V|-1 :
6     for each e (u, v)  $\in$  E:
7       Relax(u,v,w)
8   for each e (u, v)  $\in$  E:
9     if d[v] > d[u] + w(u, v)
10       return FALSE
11  return True
```



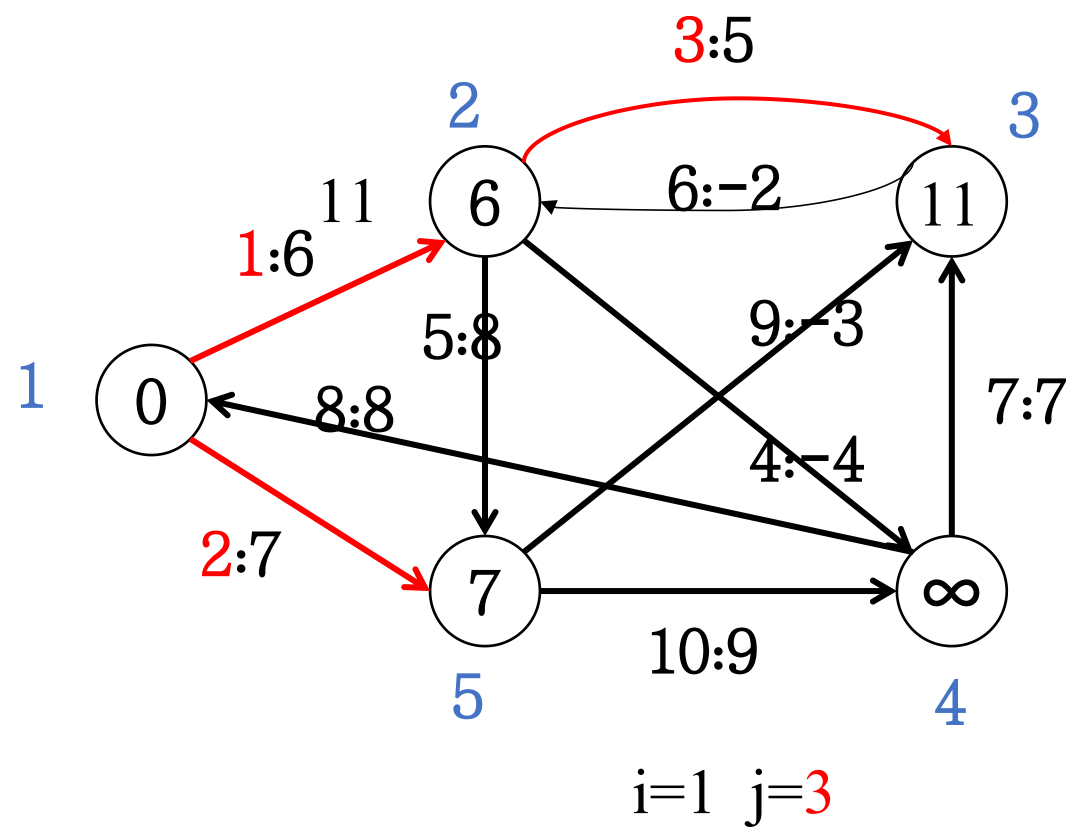
Bellman-Ford算法



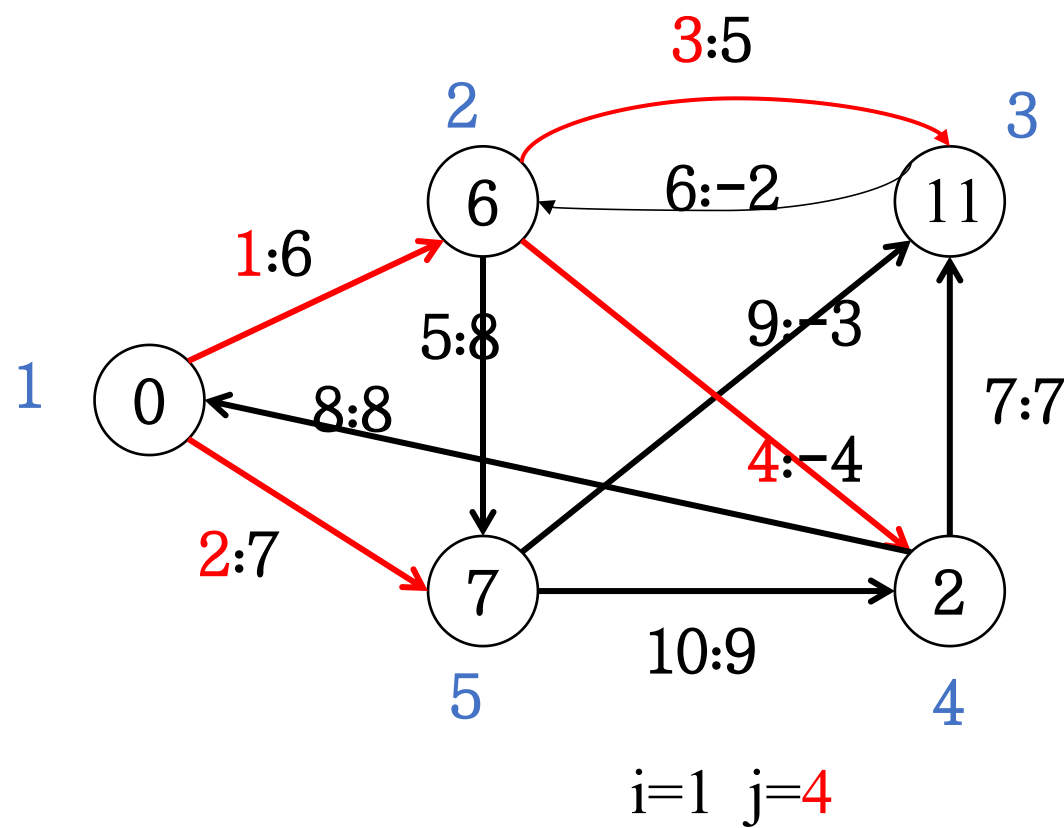
Bellman-Ford算法



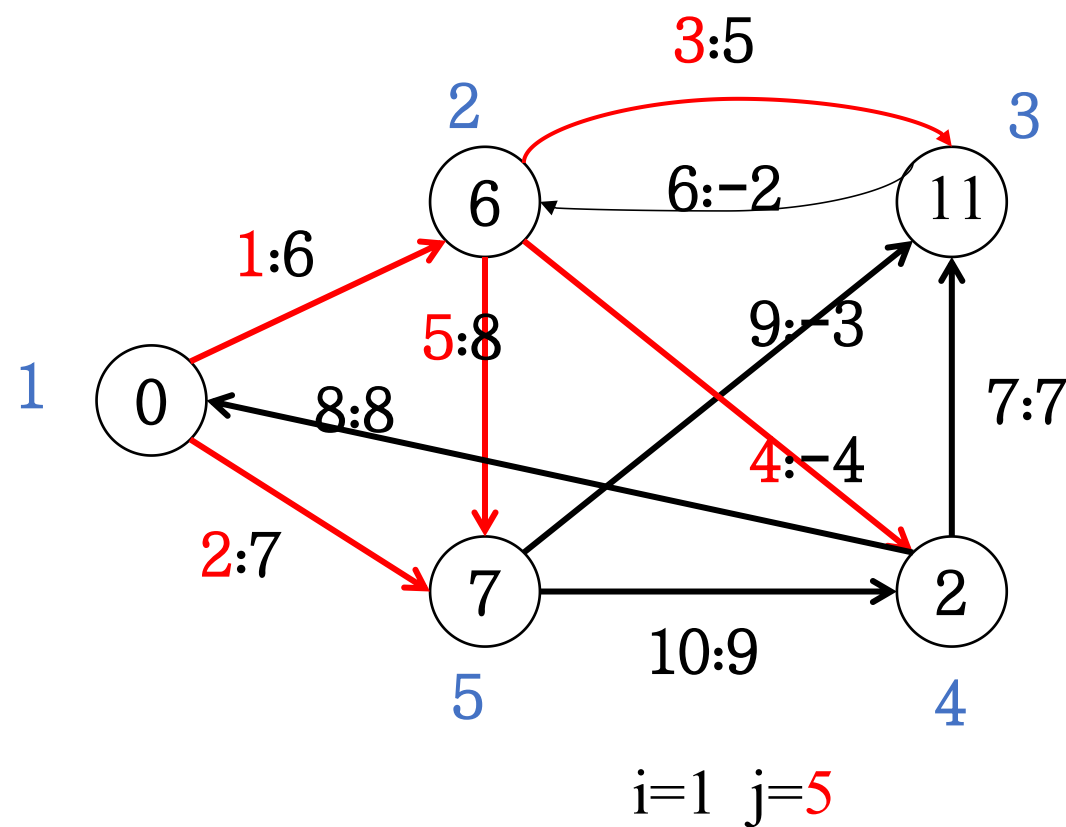
Bellman-Ford算法



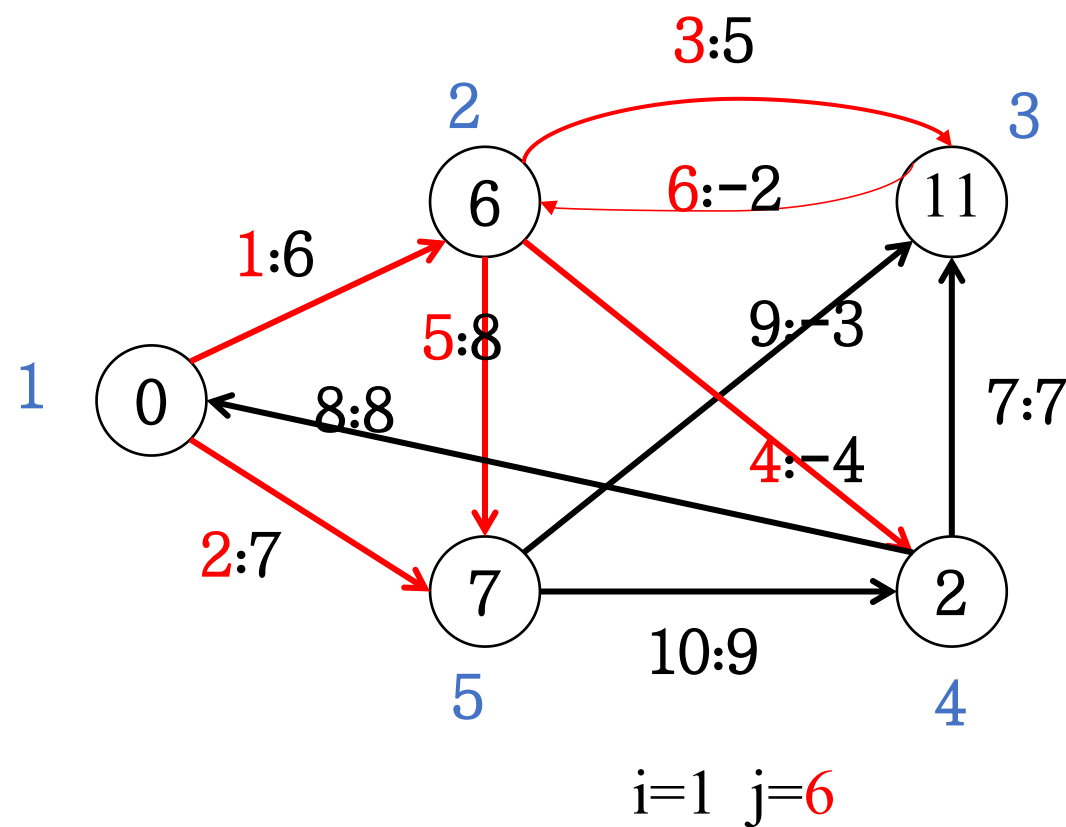
Bellman-Ford算法



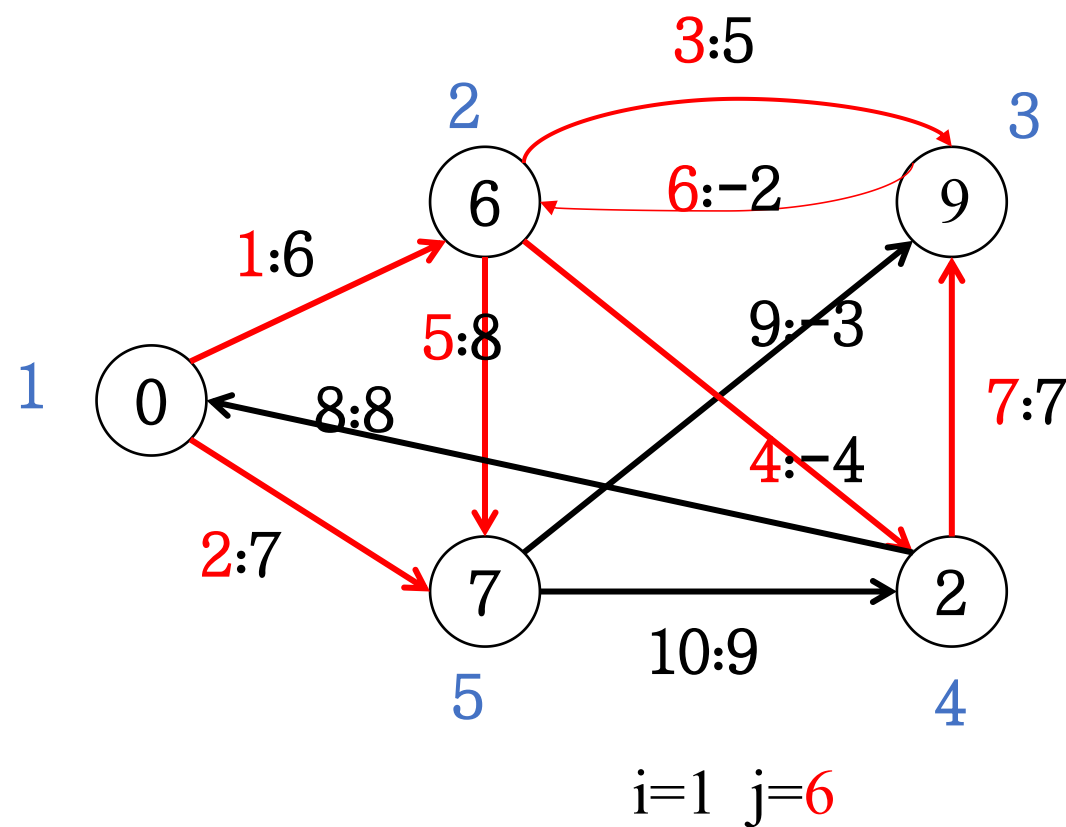
Bellman-Ford算法



Bellman-Ford算法



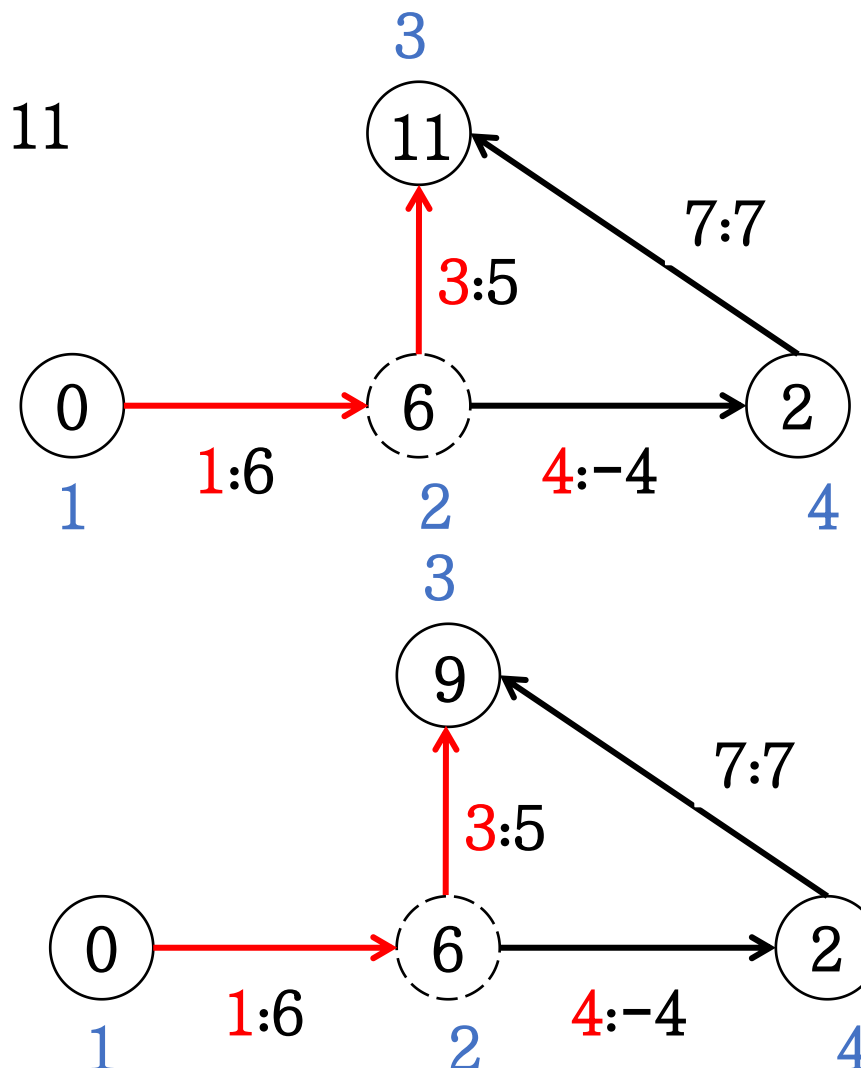
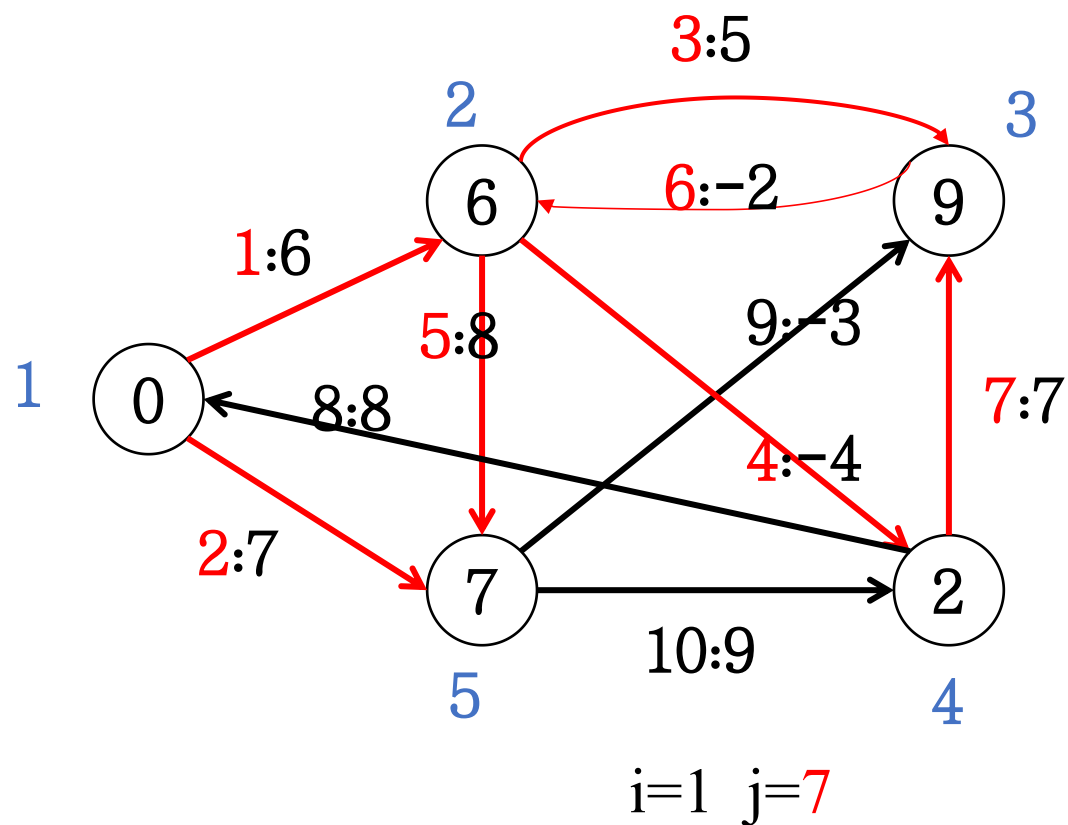
Bellman-Ford算法



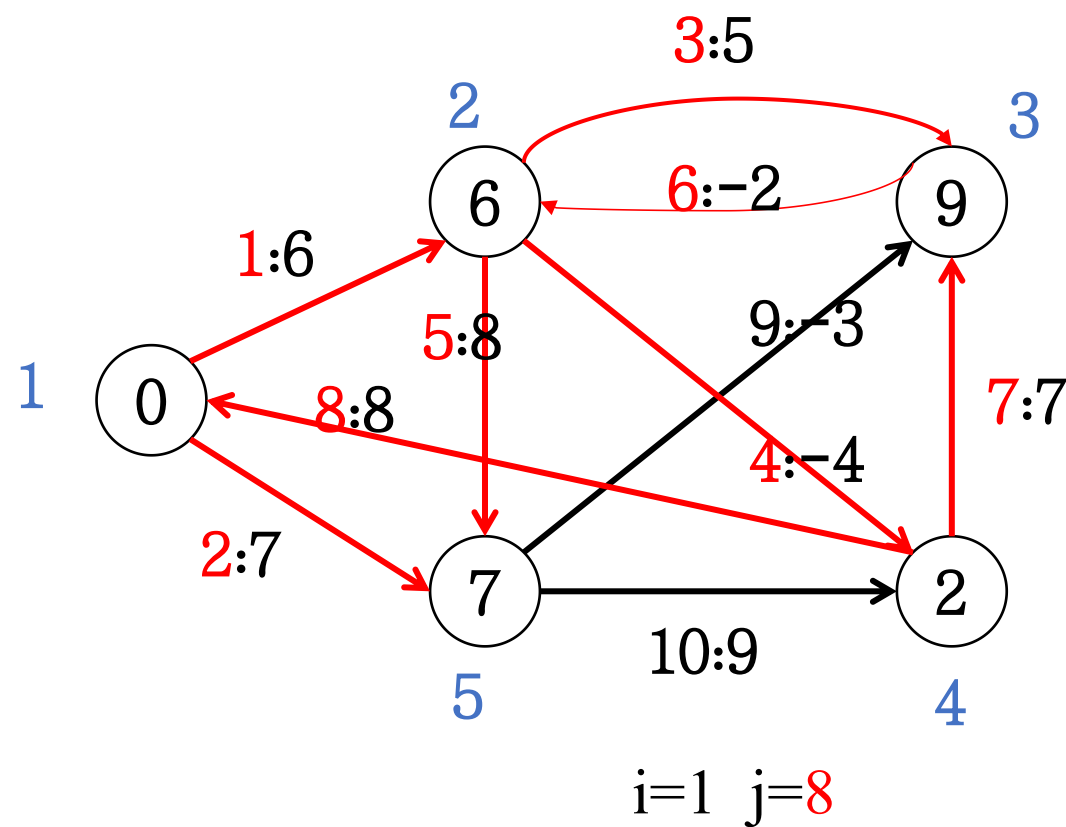
Bellman-Ford算法



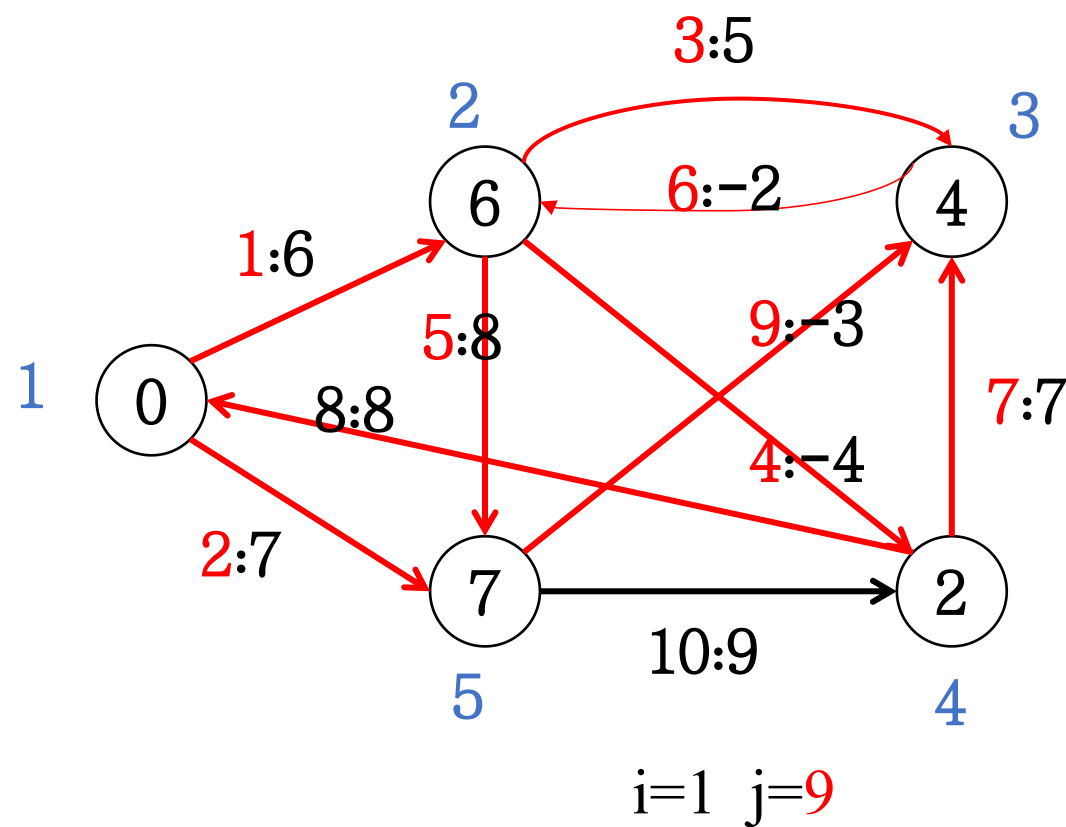
1->3目前最短路径: 11



Bellman-Ford算法



Bellman-Ford算法

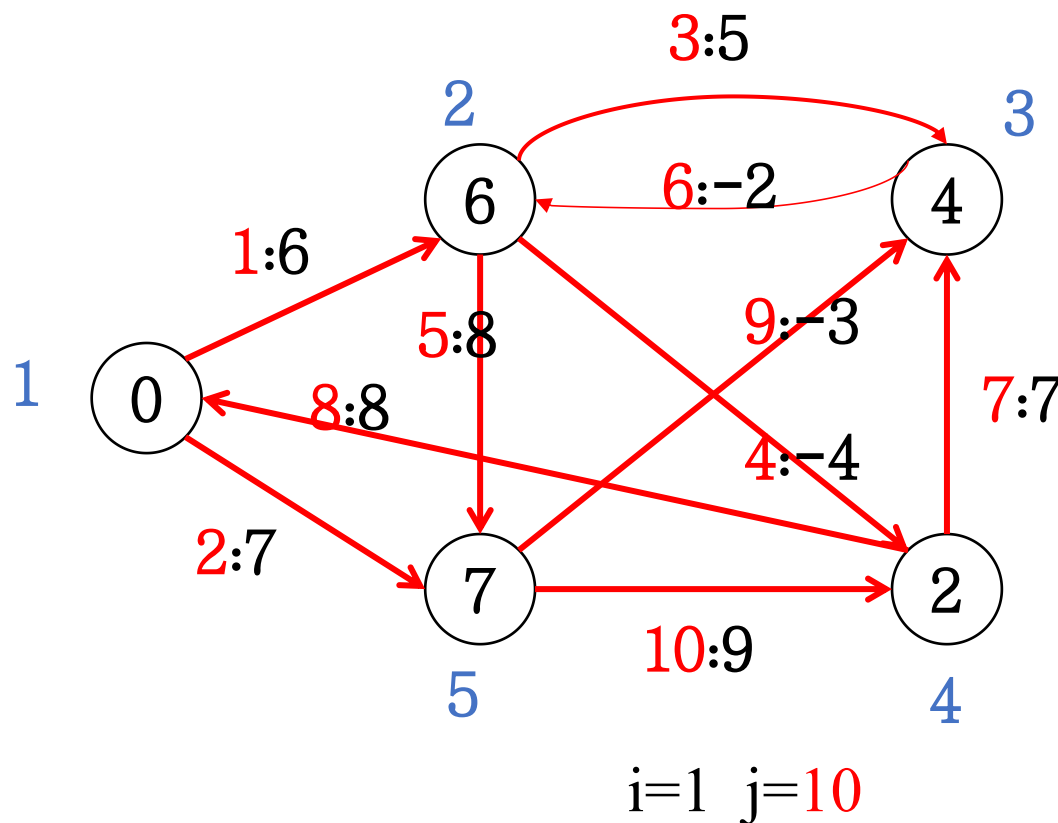


Bellman-Ford算法

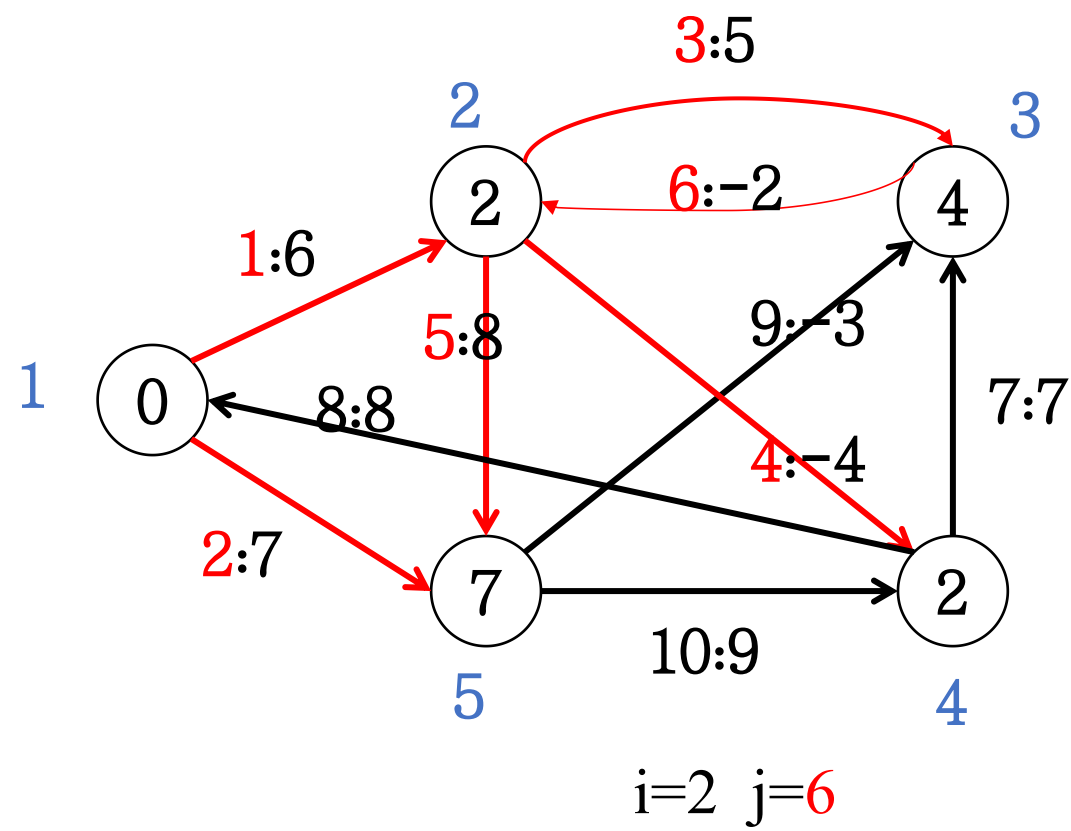
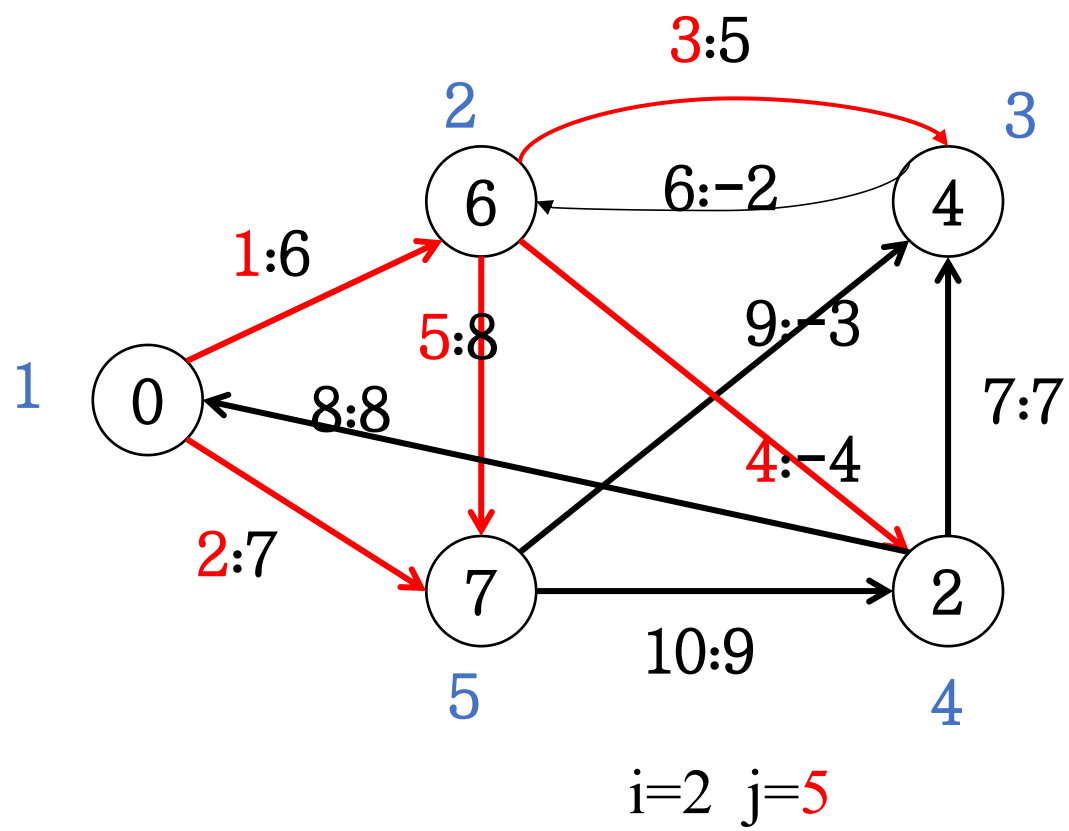


算法伪代码:

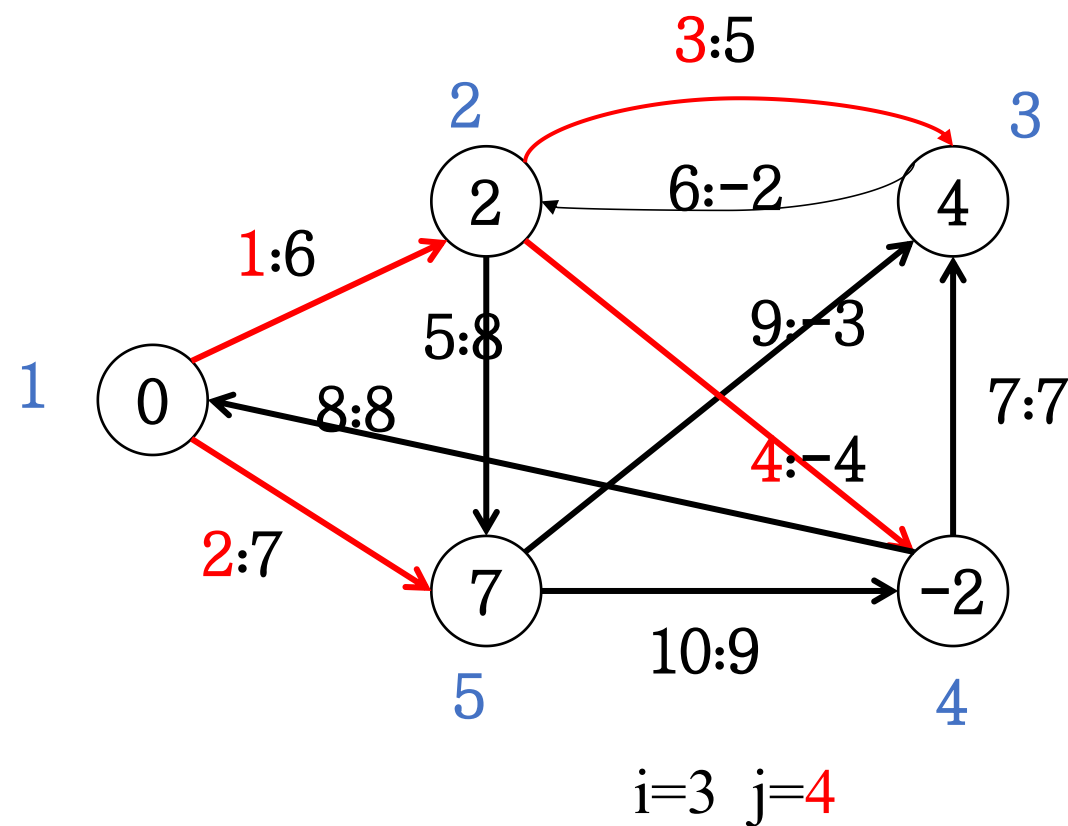
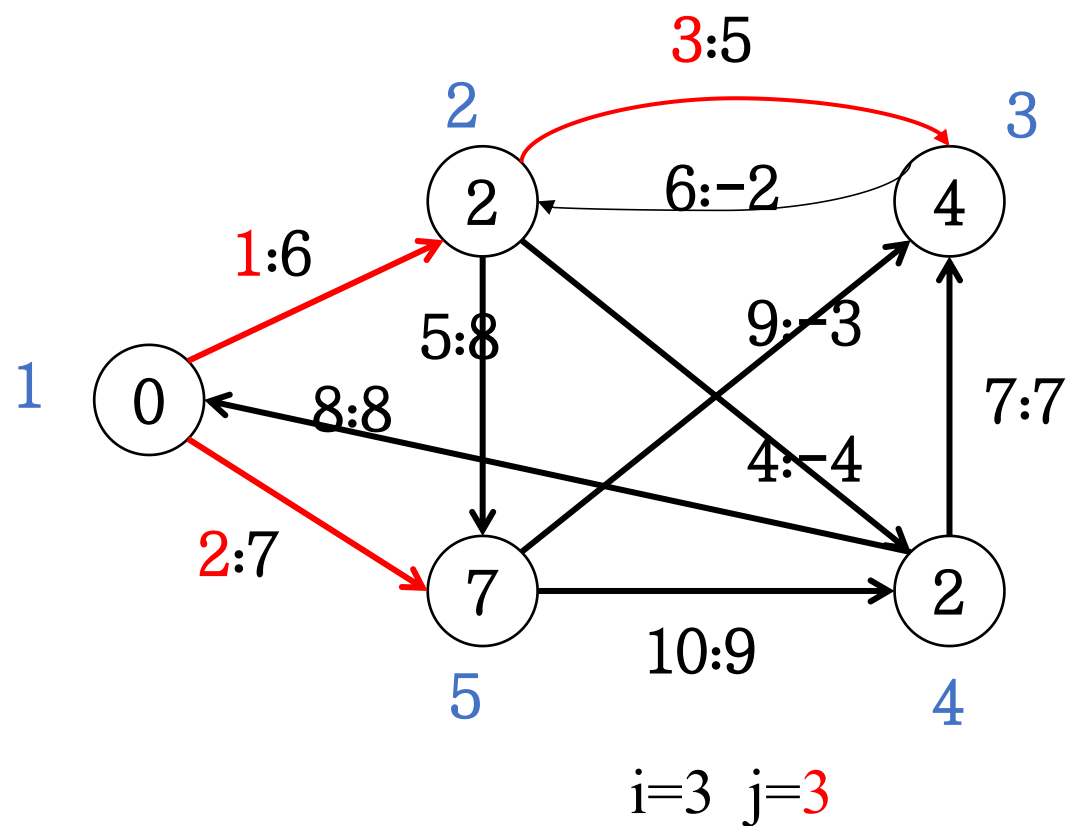
```
1 Bellman-Ford(G,w,s)
2   for v in V:
3     d[v]=  $\infty$ ; p[v]=NULL;
4     d[s]=0;
5   for i from 1 to |V|-1 :
6     for each e (u, v)  $\in$  E:
7       Relax(u,v,w)
8   for each e (u, v)  $\in$  E:
9     if d[v] > d[u] + w(u, v)
10      return FALSE
11  return True
```



Bellman-Ford算法



Bellman-Ford算法

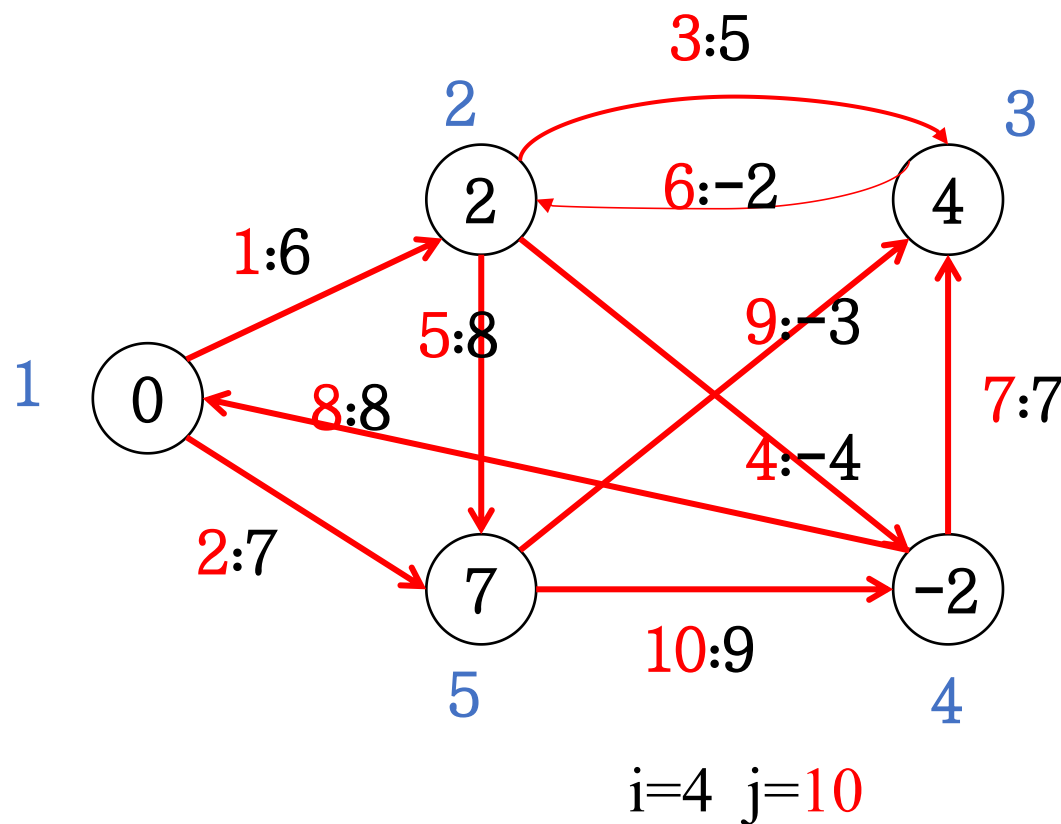


Bellman-Ford算法



算法伪代码:

```
1 Bellman-Ford(G,w,s)
2   for v in V:
3     d[v]=  $\infty$ ; p[v]=NULL;
4     d[s]=0;
5   for i from 1 to |V|-1 :
6     for each e (u, v)  $\in$  E:
7       Relax(u,v,w)
8   for each e (u, v)  $\in$  E:
9     if d[v] > d[u] + w(u, v)
10      return FALSE
11  return True
```



Bellman-Ford算法



算法时间复杂度分析:

```
1 Bellman-Ford(G,w,s)
2   for v in V:
3     d[v]=  $\infty$ ; p[v]=NULL;
4     d[s]=0;
5   for i from 1 to |V|-1 :
6     for each e (u, v)  $\in$  E:
7       Relax(u,v,w)
8   for each e (u, v)  $\in$  E:
9     if d[v] > d[u] + w(u, v)
10      return FALSE
11  return True
```

$\left. \begin{array}{l} \text{for each } e (u, v) \in E: \\ \text{Relax}(u,v,w) \end{array} \right\} O(|E|)$

$\left. \begin{array}{l} \text{for each } e (u, v) \in E: \\ \text{if } d[v] > d[u] + w(u, v) \end{array} \right\} O(|E|)$

$\left. \begin{array}{l} O(|E|) \\ O(|E|) \end{array} \right\} O(|V| * |E|)$

Bellman-Ford算法



正确性证明:

给定带权图 $G=(V, E, W)$, 如果图 G 中不包含从源点 s 可达的负权环路, 那么上述算法在 $|V|-1$ 次迭代之后对所有 s 可达的点 v 都有 $d[v]=\delta(s, v)$ 。

证明: 数学归纳法, 对于路径长度进行归纳, 证明长度为 n 的最短路径在第 n 次循环就已经得到

基础步: 长度为0的最短路径在第0次循环就已经得到, 由于没有负权环路, 初始化阶段就有 $d[s]=\delta(s, s)=0$;

归纳步: 长度小于等于 n 的最短路径在第 n 次循环就已经得到, 长度为 $n+1$ 最短路径有两部分组成: 长度为 n 的最短路径和只有一条边 e_{n+1} 的 $d[v_n]=\delta(s, v_n)$ 一条最短路径。

长度为 n 的最短路径按归纳假设有, 于是按照松弛操作有 $d[v_{n+1}]=d[v_n]+w(e_{n+1})=\delta(s, v_{n+1})$

Bellman-Ford算法



正确性证明:

如果 $d[v]$ 在Bellman-Ford算法的 $|V|-1$ 次迭代之后还能继续松弛, 那么图中有负权环路。

证明: 如果在 $|V|-1$ 次循环之后点 v 还能继续松弛, 那么从 s 到 v 的最短路径比 $|V|-1$ 还要长, 那么其中必须有个环路。

这个环路必然权重是负数, 否则不会松弛

目录

第一节

最短路径问题

第二节

松弛算法

第三节

有向无环图上的
最短路径计算

第四节

Dijkstra算法

第五节

Bellman-Ford算法

第六节

差分约束和最
短路径

第六节

路径计算前沿

差分约束和最短路径



线性规划问题:

研究线性约束条件下线性目标函数的极值问题的数学理论和方法。

定义：给定一个 $m \times n$ 维矩阵 A 、 n 维向量 c 和 m 维向量 b ，求一个 n 维变量矩阵，在 $Ax \leq b$ 的情况下，最大化 $\sum_{i=1}^n c_i x_i$

差分约束和最短路径



差分约束系统:

对于线性规划问题而言，如果矩阵A中每一行只有一个1和-1且其它值为0。因此，由 $Ax \leq b$ 所给出的约束条件变为m个涉及n个变量的差额限制条件，其中每个约束条件是如下所示的简单线性不等式:

$$x_i - x_j \leq b_k$$

这里 $1 \leq i, j \leq n, 1 \leq k \leq m$

差分约束和最短路径



差分约束系统:

比如对于下列问题，求解合法的 $\mathbf{x} = (x_i)$

$$\begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} \leq \begin{bmatrix} 0 \\ -1 \\ 1 \\ 5 \\ 4 \\ -1 \\ -3 \\ -3 \end{bmatrix}$$

差分约束和最短路径



差分约束系统:

$$x_1 - x_2 \leq 0$$

$$x_1 - x_5 \leq -1$$

$$x_2 - x_5 \leq 1$$

$$x_3 - x_1 \leq 5$$

$$x_4 - x_1 \leq 4$$

$$x_4 - x_3 \leq -1$$

$$x_5 - x_3 \leq -3$$

$$x_5 - x_4 \leq 3$$

差分约束和最短路径



差分约束系统:

$$x_1 - x_2 \leq 0$$

$$x_1 - x_5 \leq -1$$

$$x_2 - x_5 \leq 1$$

$$x_3 - x_1 \leq 5$$

$$x_4 - x_1 \leq 4$$

$$x_4 - x_3 \leq -1$$

$$x_5 - x_3 \leq -3$$

$$x_5 - x_4 \leq 3$$

可能解:

$$x = (-5, -3, 0, -1, -4)$$

$$x' = (0, 2, 5, 4, 1)$$

差分约束



引理1:

如果存在一组解 $\{x_1, x_2, \dots, x_n\}$ 的话, 那么对于任何一个常数 d 有 $\{x_1+d, x_2+d, \dots, x_n+d\}$ 也肯定是一组解, 因为任何两个数加上一个数以后, 它们之间的关系 (差) 是不变的, 这个差分约束系统中的所有不等式都不会被破坏。

证明见算法导论P388

差分约束和图



- 可以从图论的视角来理解差分约束系统，将其建模为一个包含 n 个顶点和 m 条边的图结构。图中的每一个顶点 v_i (其中 $i=1,2,\dots,n$) 都精确对应于系统中的一个未知变量 x_i 。而图中的每一条有向边，则代表了 m 个线性不等式约束中的一个。
- 具体而言，对于形如 $x_j - x_i \leq b_k$ 的约束条件，在图中建立一条从顶点 v_i 指向顶点 v_j 的有向边，从而将代数约束转化为图的拓扑结构。

差分约束和图



$$x_1 - x_2 \leq 0$$

$$x_1 - x_5 \leq -1$$

$$x_2 - x_5 \leq 1$$

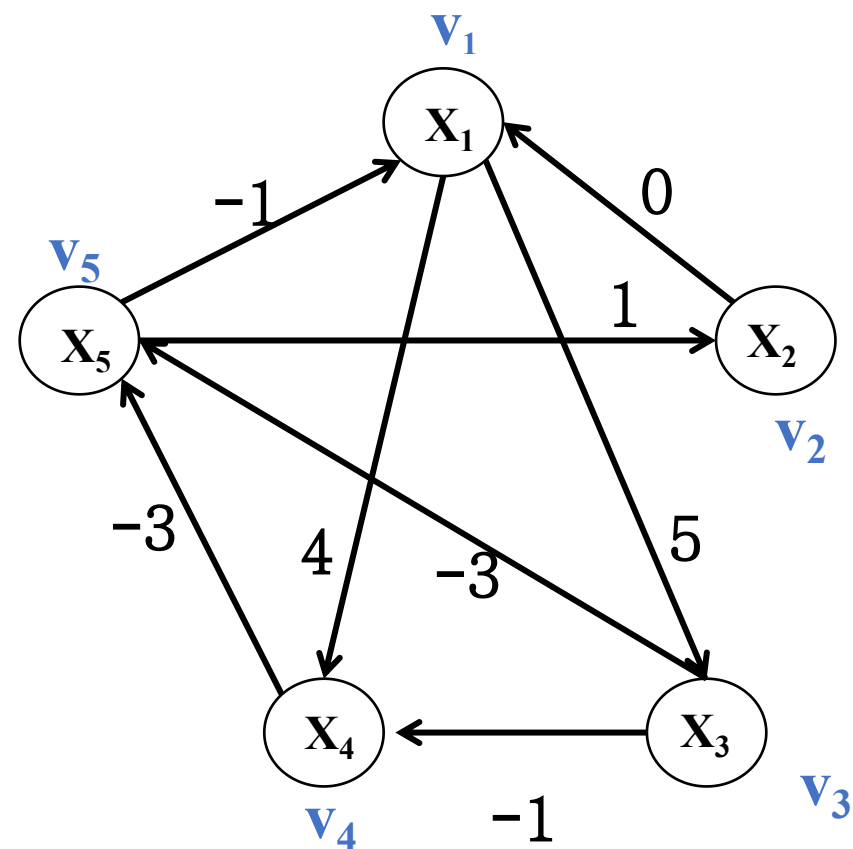
$$x_3 - x_1 \leq 5$$

$$x_4 - x_1 \leq 4$$

$$x_4 - x_3 \leq -1$$

$$x_5 - x_3 \leq -3$$

$$x_5 - x_4 \leq 3$$



差分约束和最短路径



限制图:

此外增加一个点 v_0 到所有点都有一个长度为0的边

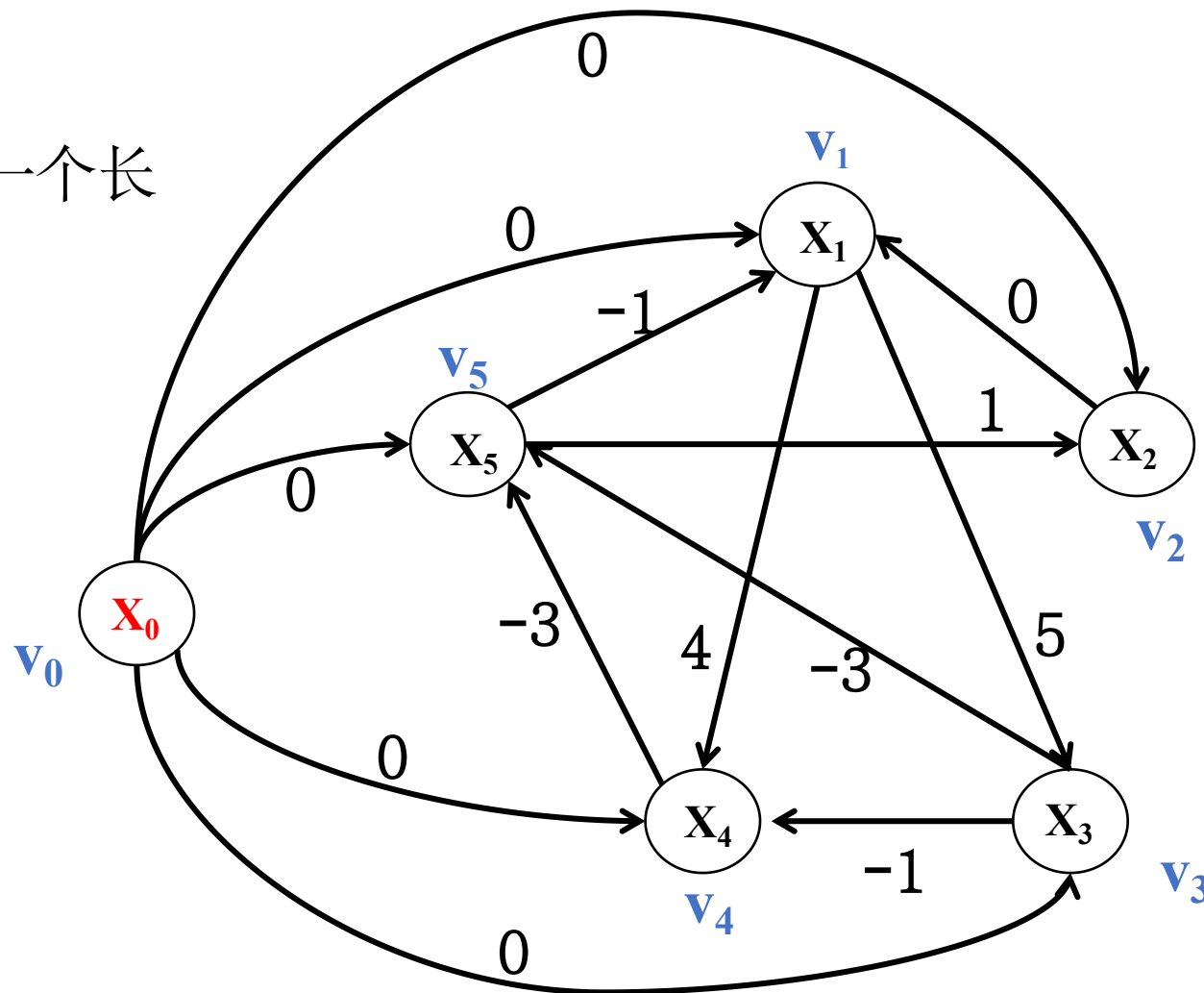
$$x_1 - x_0 \leq 0$$

$$x_2 - x_0 \leq 0$$

$$x_3 - x_0 \leq 0$$

$$x_4 - x_0 \leq 0$$

$$x_5 - x_0 \leq 0$$



差分约束和最短路径



定理（算法导论P389）：

如果限制图中没有负权环路，那么上述算法求出 v_0 到所有点的最短路径距离就是一个差分约束系统的解。

证明：对于任意一条限制图上的边 (x_i, x_j) ，根据三角不等式，我们有 $\delta(s, x_i) + w_{ij} \geq \delta(s, x_j)$ ，于是 $\delta(s, x_i) - \delta(s, x_j) \leq w_{ij}$ ，即差分约束

差分约束和最短路径



定理（算法导论P389）：

如果限制图G中有负权环路，那么这个差分约束系统没有结果

证明：假设存在一条负权环路 $v_0 e_1 v_1 e_2 \dots e_n v_n e_{n+1} v_0$ 且 x_i 对应 v_i ，那么

$$x_0 - x_1 \leq w_{01}$$

$$x_1 - x_2 \leq w_{12}$$

.....

$$x_n - x_0 \leq w_{n0}$$

上面的不等式左右分别求和，就有 $0 \leq w_{01} + w_{12} + \dots + w_{n0}$ ，由于 $v_0 e_1 v_1 e_2 \dots e_n v_n e_{n+1} v_0$ 是负权环路，那么 $w_{01} + w_{12} + \dots + w_{n0} < 0$ 。

差分约束和最短路径



求解差分约束系统:

基于Bellman-Ford算法的算法。首先，在限制图 G 的基础上，构造一个图 G' ，引入一个源点 s ， s 到所有点都有一条边，长度都为0

然后，对 G' 进行Bellman-Ford算法得到 s 到所有点的最短路径距离

这些距离就是 $d[x_i]$ 就是一个解

复杂度为 $O(m*n)$

目录

第一节

最短路径问题

第二节

松弛算法

第三节

有向无环图上的
最短路径计算

第四节

Dijkstra算法

第五节

Bellman-Ford算法

第六节

差分约束和最
短路径

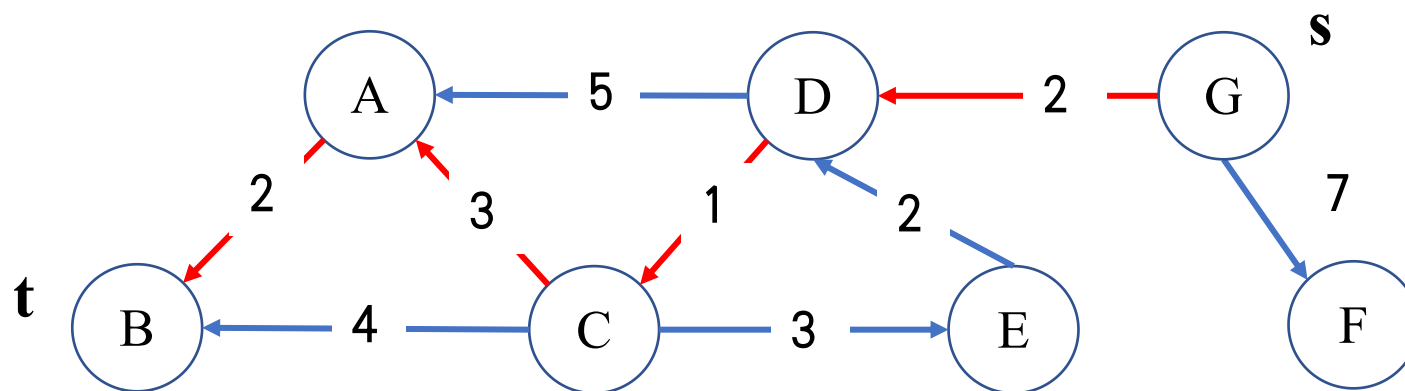
第六节

路径计算前沿

两点间最短路径



给定一个带权图 $G=(V, E, W)$ 和两个点 s 和 t ，计算 G 上从 s 到 t 的距离 $\delta(s, t)$



基本算法



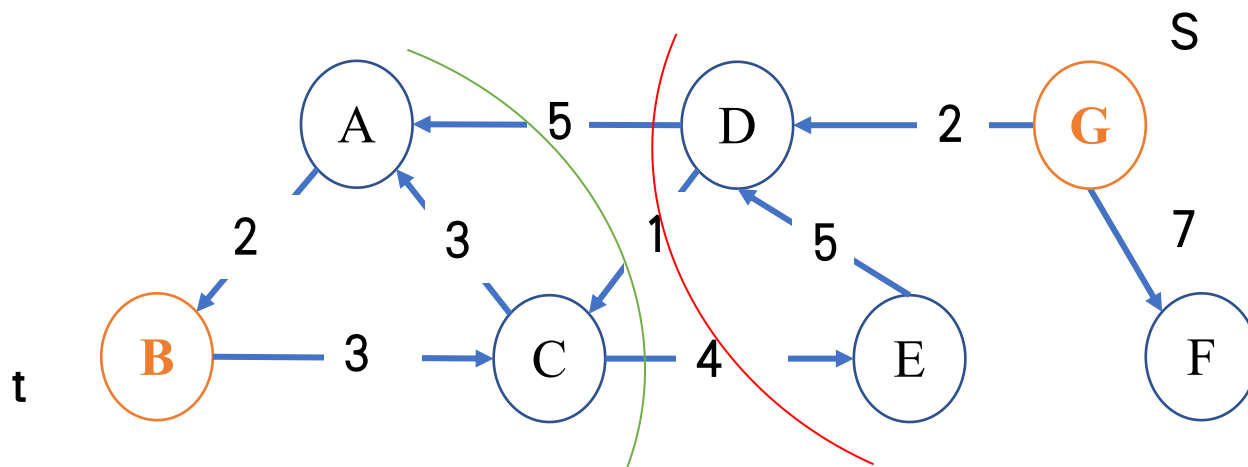
```
1 Dijkstra(G, Adj, s)
2 Initialize();
3 Q = V
4 while Q ≠ ∅;
5     u = EXTRACT-MIN(Q); A = A ∪ {u};
6     if u == t
7         return  $\delta(s, t)$ 
8     for each v ∈ Adj[u]:
9         Relax(u, v, G)
```

遇到t的时候
直接终止

双向Dijkstra算法



给定一个带权图 $G=(V, E, W)$ 和两个点 s 和 t ，从 s 向前做搜索（Forward Search）的同时交替地进行从 t 开始的反向搜索（Backward Search）



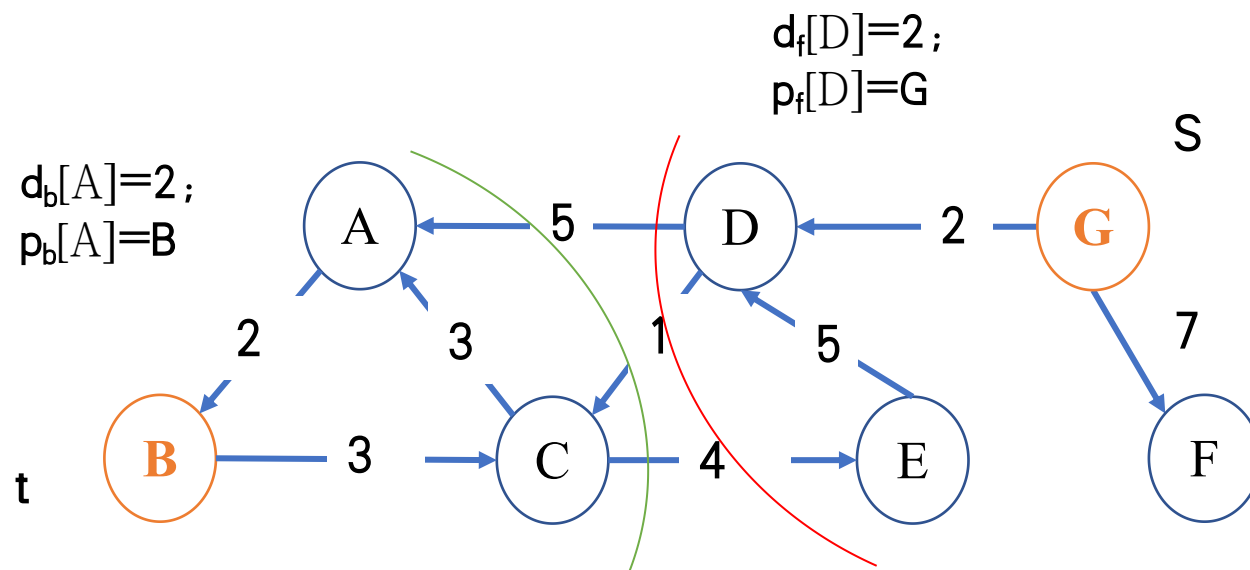
相关定义



给定一个点 v , $d_f[v]$ 和 $d_b[v]$ 定义为前向搜索和后向搜索过程中 s 到 v 和 v 到 t 的距离; $p_f[v]$ 和 $p_b[v]$ 定义为前向搜索和后向搜索过程中 s 到 v 和 v 到 t 的路径上的前驱结点; Q_f 和 Q_b 分别为前向搜索和后向搜索过程中优先队列

$$Q_b = \{A\}$$

$$Q_f = \{D\}$$



算法中止条件



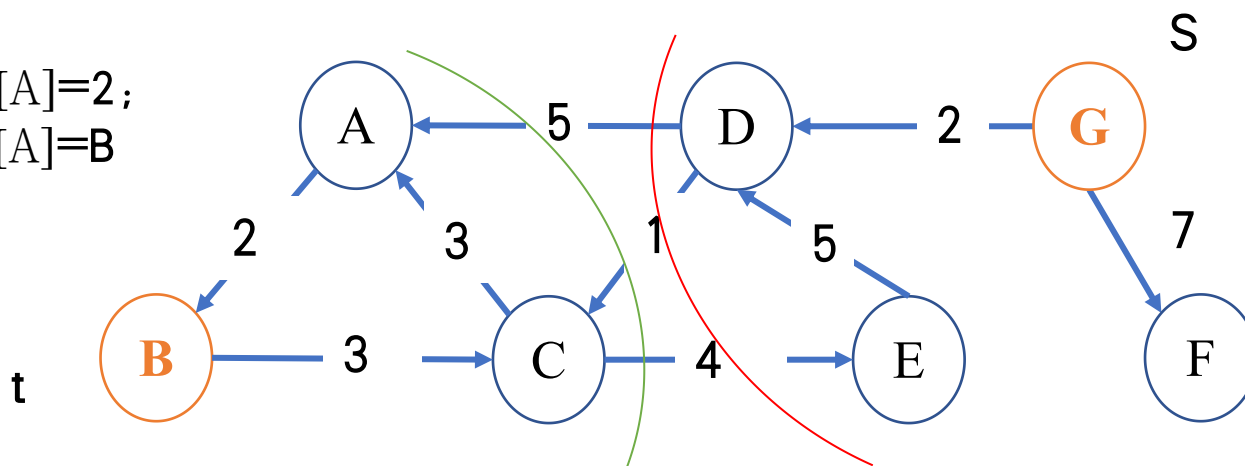
存在一个点 w 从 Q_f 和 Q_b 中都被删除的时候，算法停止

$Q_b = \{C, D\}$

$d_f[D] = 2;$
 $\pi_f[D] = G$

$Q_f = \{C, A, F\}$

$d_b[A] = 2;$
 $p_b[A] = B$

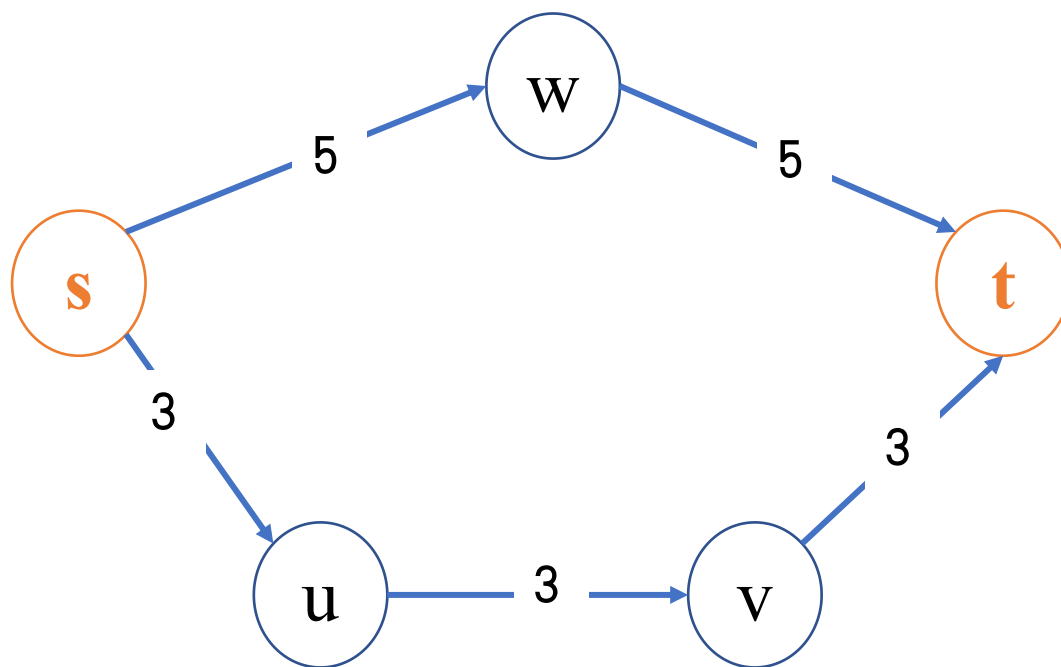


$d_f[C] = 3; d_b[C] = 5$
 $p_f[C] = D; p_b[C] = A$

最终解计算



在点 w 从 Q_f 和 Q_b 中都被删除的时候，直接从 w 出发得到最终解是不行的



正确的最终解



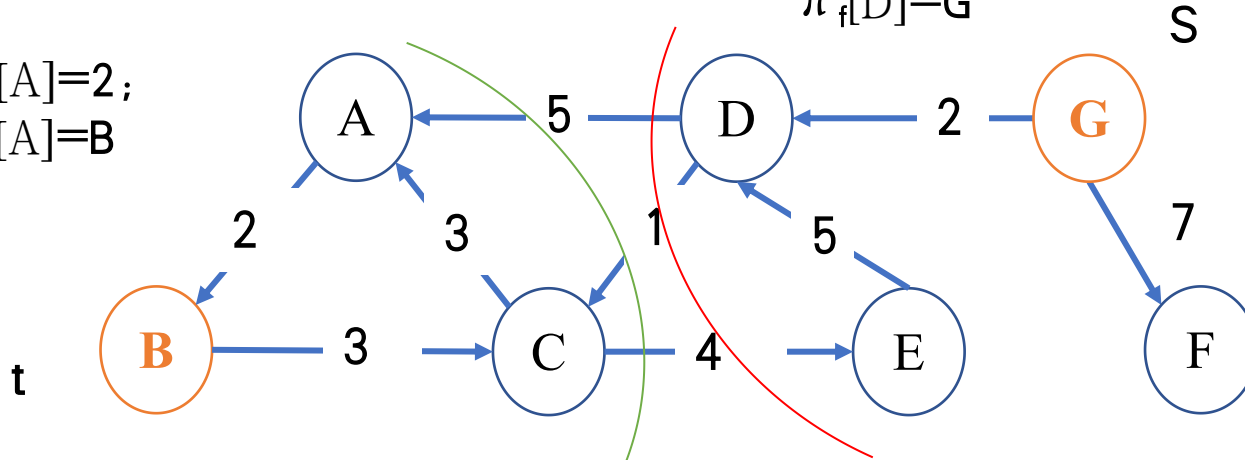
正确的最终解计算方法应该是从 Q_f 和 Q_b 中找出连接s和t的最小路径来返回，也就是返回： $\min_x \{d_f[x] + d_b[x]\}$

$Q_b = \{C, D\}$

$d_b[A] = 2;$
 $p_b[A] = B$

$d_f[D] = 2;$
 $\pi_f[D] = G$

$Q_f = \{C, A, F\}$



$d_f[C] = 3; d_b[C] = 5$
 $p_f[C] = D; p_b[C] = A$

基于Landmark的优化



可以按照之前的边重赋权方法对所有边进行重赋权，即找出一个函数 h ，对于任意边 (u,v) ，我们有

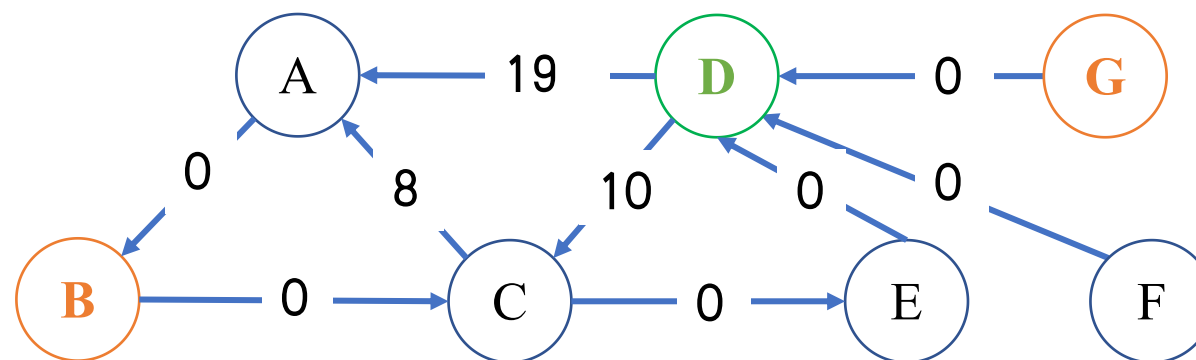
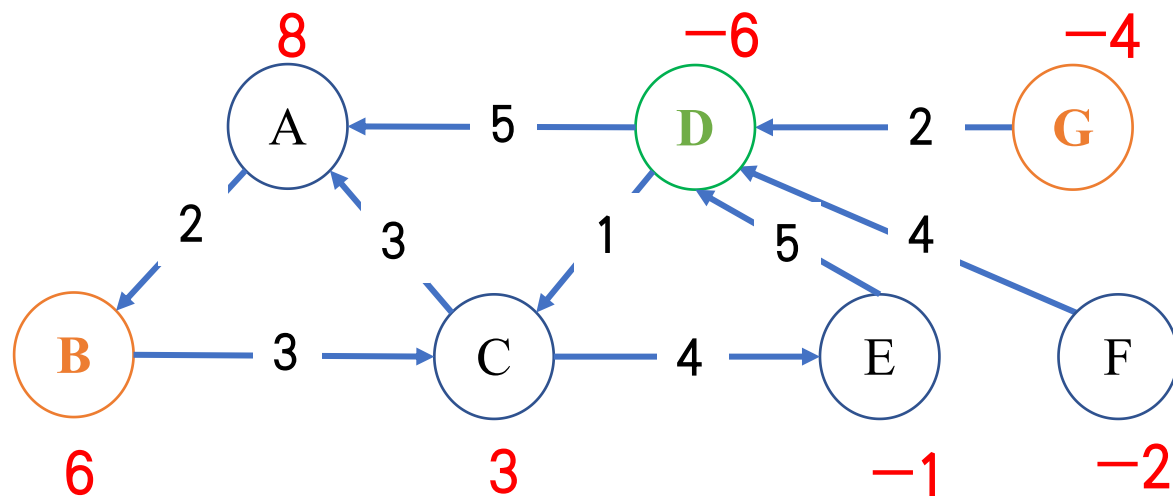
$$w_h(u, v) = w(u, v) + h(v) - h(u)$$

之后，找出一个landmark l 来定义 h 以加快Dijkstra算法计算过程，即

$$h(u) = \delta(u, l) - \delta(l, t)$$

所有 $\delta(u, l)$ 和 $\delta(l, t)$ 可以在预处理阶段事先算好

基于Landmark的优化



基于Landmark的距离估计



Michalis Potamias, Francesco Bonchi, Carlos Castillo, Aristides Gionis: Fast shortest path distance estimation in large networks. CIKM 2009: 867-876

Andrey Gubichev, Srikanta J. Bedathur, Stephan Seufert, Gerhard Weikum: Fast and accurate estimation of shortest paths in large graphs. CIKM 2010: 499-508

Miao Qiao, Hong Cheng, Lijun Chang, Jeffrey Xu Yu: Approximate Shortest Distance Computing: A Query-Dependent Local Landmark Scheme. ICDE 2012: 462-473

Mingdao Li, Peng Peng, Yang Xu, Hao Xia, Zheng Qin: Distributed Landmark Selection for Lower Bound Estimation of Distances in Large Graphs. APWeb/WAIM (1) 2019: 223-239

问题定义



给定一个无向图 $G=(V, E)$ ，所谓landmark，就是一些特定的点。我们记录并保存所有点到landmark的距离，进而对于每个点 v 形成一个向量

$$\phi(v) = \langle \delta(v, l_1), \delta(v, l_2), \dots, \delta(v, l_n) \rangle$$

对于两个查询点 s 和 t 而言，显然有

$$\delta_{\text{approx}}(s, t) = \min_{1 \leq k \leq n} \{ \delta(s, l_k) + \delta(t, l_k) \} \geq \delta(s, t)$$

$$\delta_{\text{approx}}(s, t) = \max_{1 \leq k \leq n} \{ | \delta(s, l_k) - \delta(t, l_k) | \} \leq \delta(s, t)$$



本文主要讨论了lankmark的选取策略问题

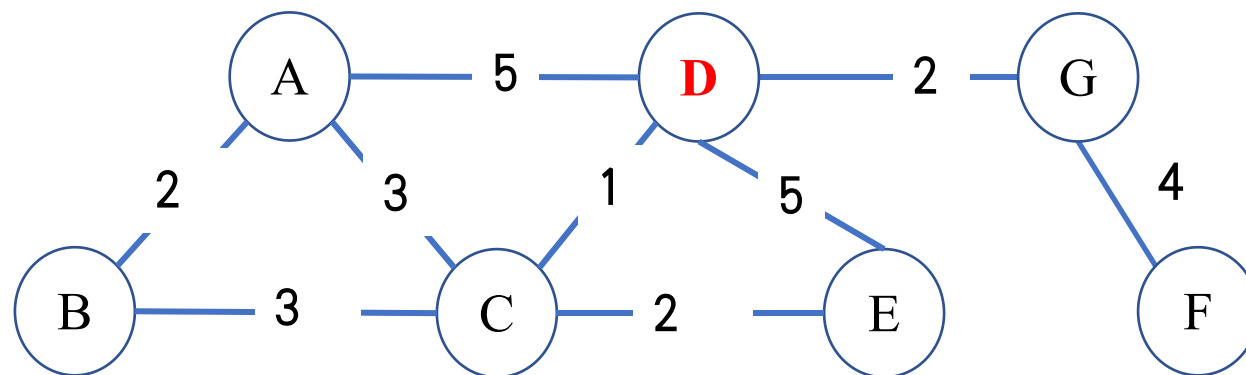
本文着重介绍了基于度中心度和介数中心度的两种策略

度中心度



度中心度 (Degree Centrality) 通过点的邻居数目来计算节点的重要程度

给定图 $G = (V, E)$, 点 v 的度中心度计算公式为: $C_D = \deg(v)$

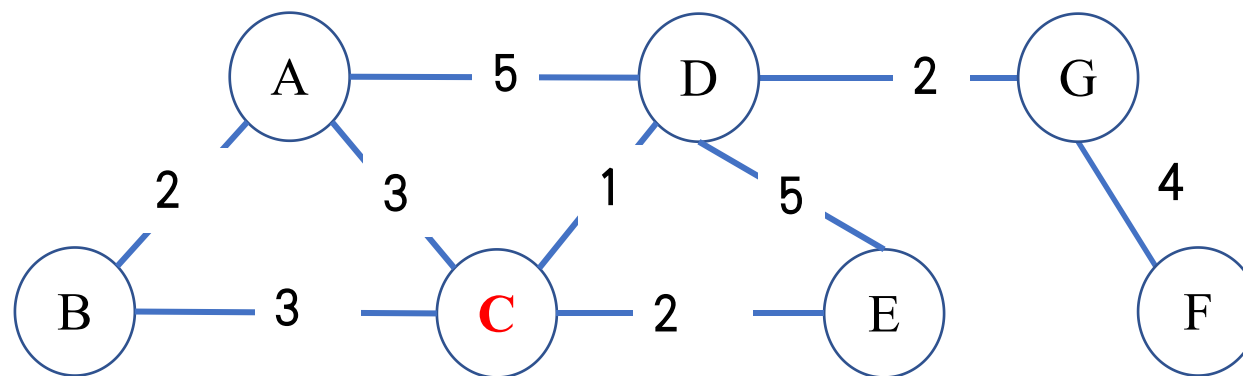


介数中心度



介数中心度 (Betweenness Centrality) 定义为 G 中除了 v 以外的点对间最短路径有多少条最短路径经过了点 v

介数中心度的计算公式为: $C_B = \sum_{s \neq v \neq t \in V} \frac{\pi_{st}(v)}{\pi_{st}}$, 其中 π_{st} 表示 s 到 t 的最短路径, $\pi_{st}(v)$ 表示 s 到 t 经过 v 的最短路径

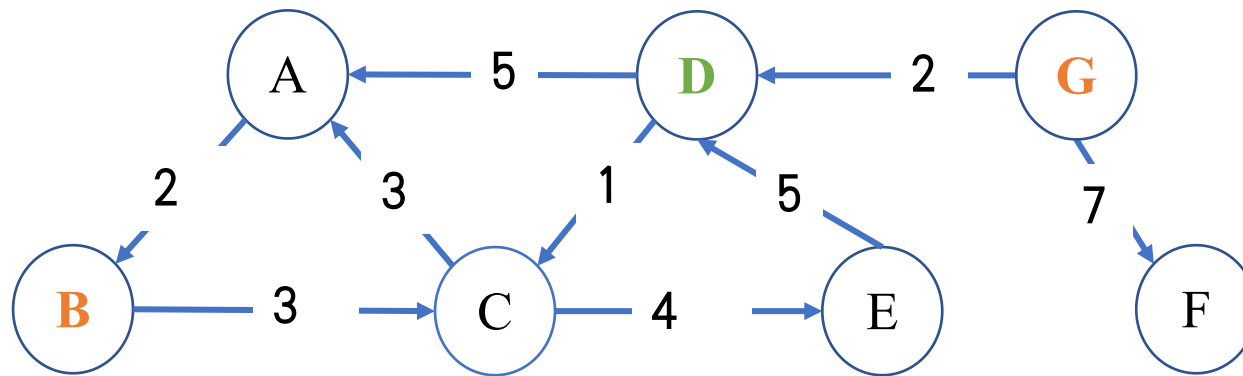




本文主要讨论了基于landmark进一步提高距离预测准确度的方法

在预处理阶段，预计算出所有Landmark的最短路径树

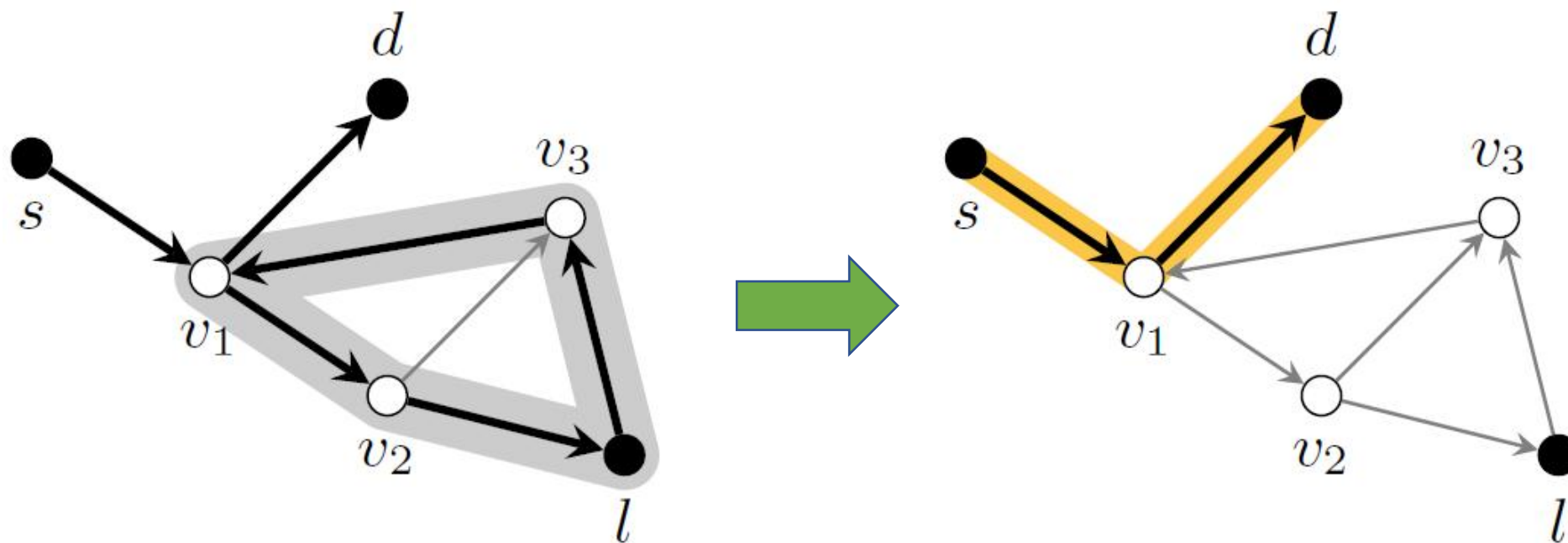
在查询处理阶段， Π_{st} 就等于 Π_{sl} 和 Π_{lt} 的拼接



Cycle Elimination



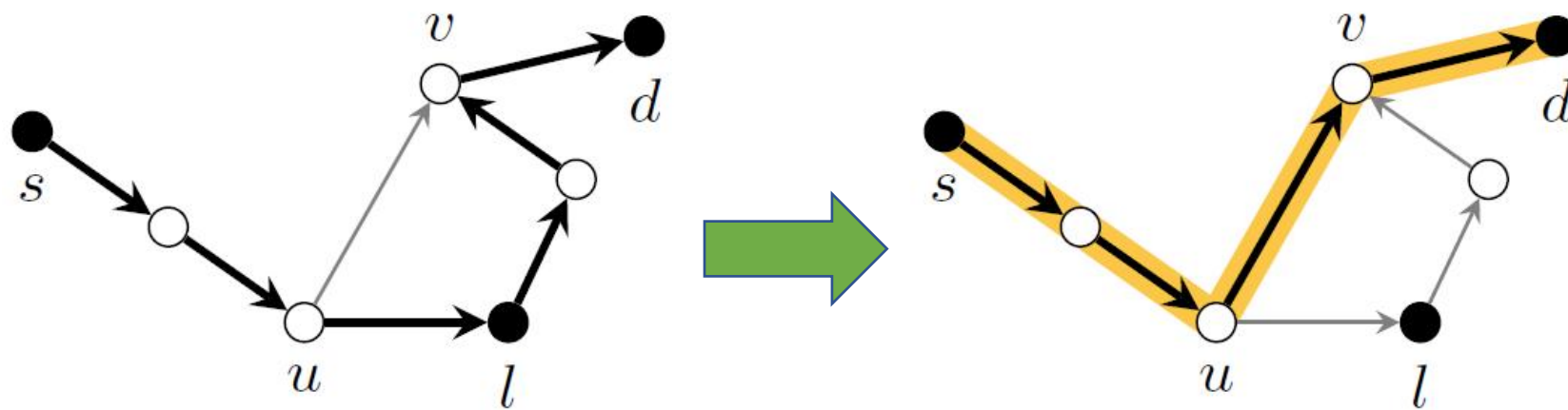
Π_{sl} 和 Π_{lt} 的拼接过程中消除期间的环



Shortcutting



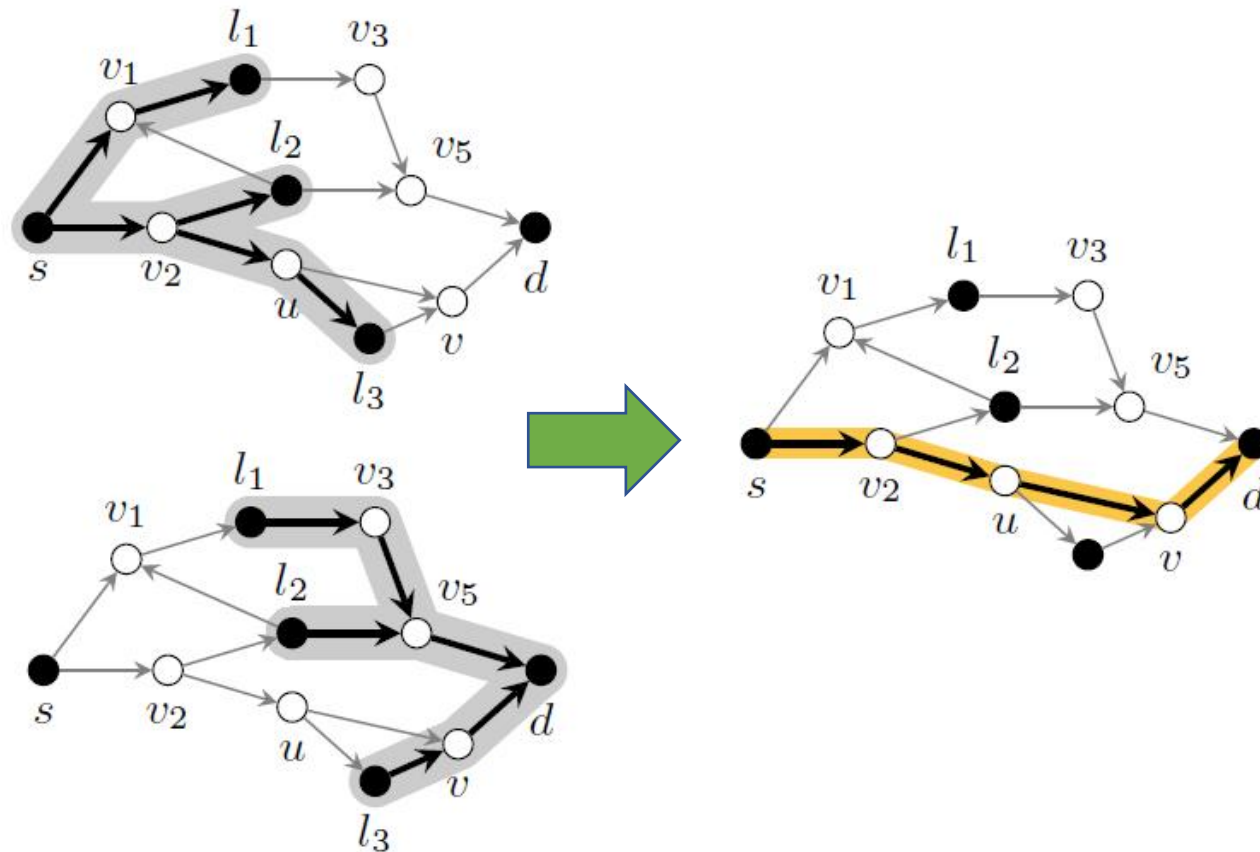
Π_{sl} 和 Π_{lt} 的拼接过程中找出期间的捷径



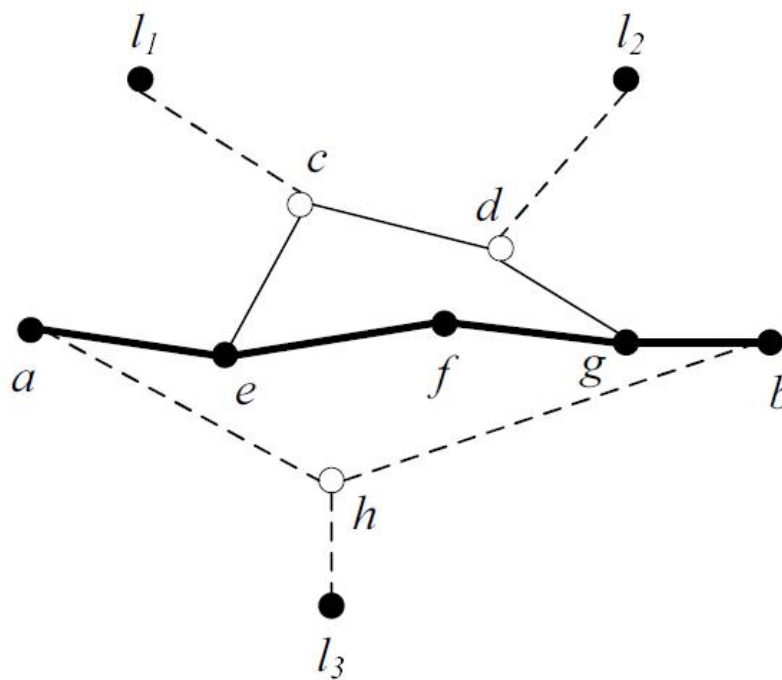
Tree Algorithm



Π_{sl} 和 Π_{lt} 的拼接过程中从最短路径树上找出跨树的路径



本文再进一步讨论了如何进一步构建索引提高效率
具体而言，本文重点讨论了基于最短路径树上最近公共祖先的优化



基于点覆盖的最短路径树计算

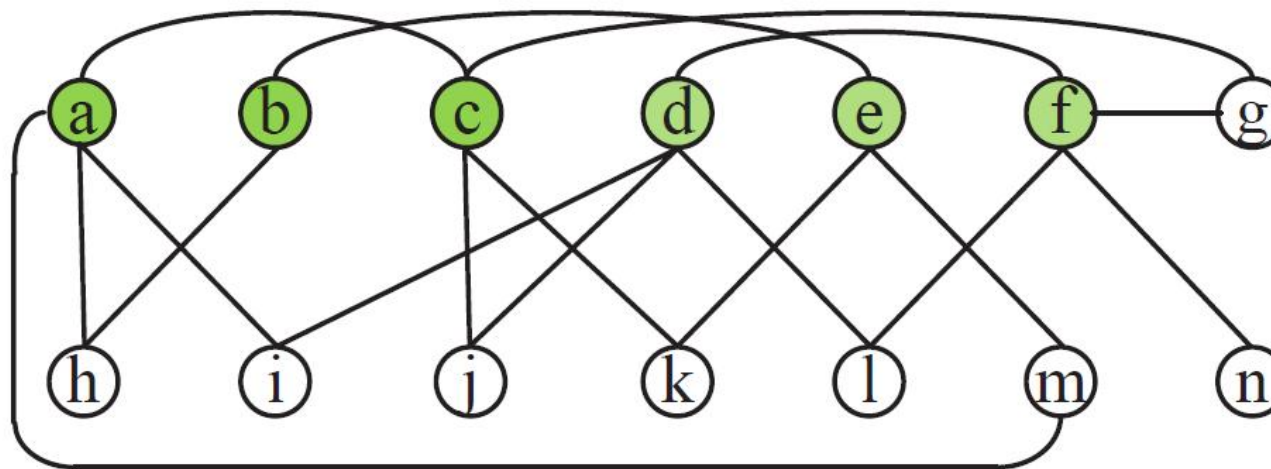


James Cheng, Yiping Ke, Shumo Chu, Carter Cheng. Efficient processing of distance queries in large graphs: a vertex cover approach. SIGMOD 2012. 457-468

点覆盖



对于无向图 $G=(V,E)$ 中的一个点覆盖是一个集合 $C\subseteq V$ 使得每一条边至少有一个端点在 C 中



点覆盖性质



性质1: 给定 $G=(V, E)$ 的一个点覆盖 C , V 中任意一个点 v 要么属于 C , 要么所有邻居都属于 C ;

性质2: 如果 G 中两点 u 和 v 之间存在一条路径 p , p 中除了 u 和 v 以外的所有点都不在 C 中, 那么 p 长度最多为2

基本算法



给定图 $G=(V,E)$ 的一个点覆盖 C ，计算出 C 中任意两点距离存在硬盘上

当查询点 s 进来后，根据性质1计算出 s 所有点的距离

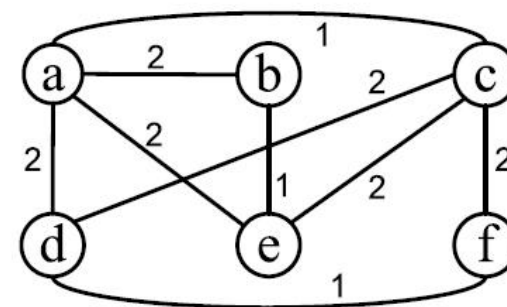
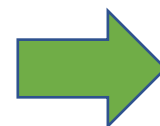
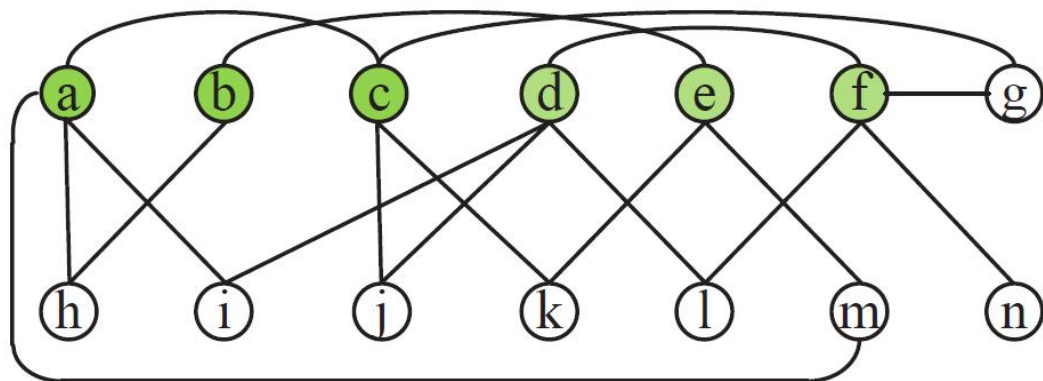
缺点：空间代价过大

改进算法



给定图 $G=(V, E)$ 的点覆盖 C ，根据性质2，将图 G 压缩成图 G'

G' 中点都是 C 中点，如果 C 中两个点 u 和 v 在 G 中两步可达， u 和 v 之间有条边，且边长为 u 和 v 之间距离



改进算法



于是，不需要存储 C 中任意两点距离，而是在计算源点 s 到所有点距离的时候用Dijkstra计算 G' 中两点间距离

G' 中点的数量小于 G 中点数量，所以 G' 上Dijkstra更快

2-Hop Labeling

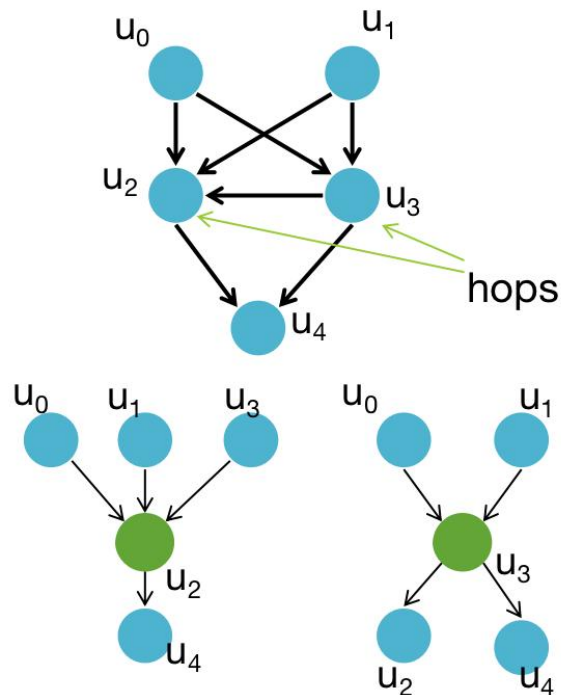


Edith Cohen, Eran Halperin, Haim Kaplan, Uri Zwick. Reachability and Distance Queries via 2-Hop Labels. SIAM J. Comput. 32(5): 1338-1355 (2003)

示例



A 2-hop distance labeling assigns to each vertex v a label $L(v) = \{L_{in}(v), L_{out}(v)\}$, such that $L_{in}(v)$ is a collection of vertices that v can reach, and similarly, $L_{out}(v)$ is a collection of vertices that can reach v

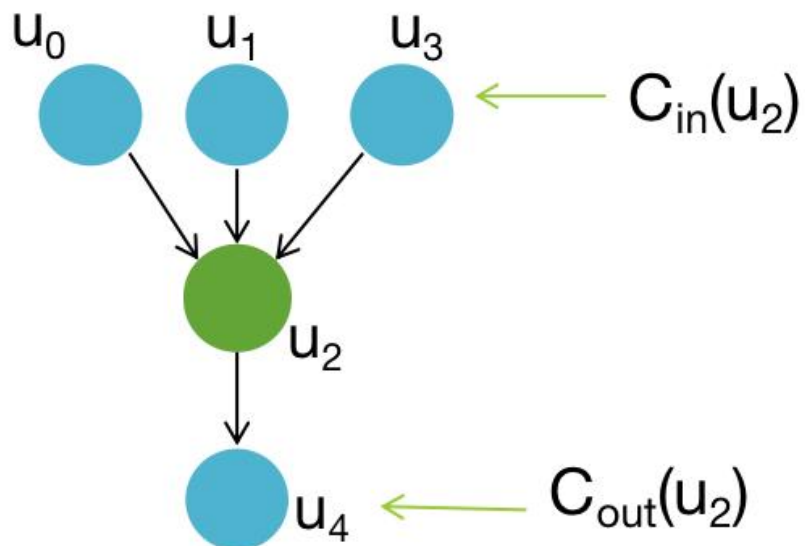


vertex	2-hop Labeling
u0	$L_{in}(u_0) = \{\}, L_{out}(u_0) = \{u_2, u_3\}$
u1	$L_{in}(u_1) = \{\}, L_{out}(u_1) = \{u_2, u_3\}$
u2	$L_{in}(u_2) = \{u_2, u_3\}, L_{out}(u_2) = \{u_2\}$
u3	$L_{in}(u_3) = \{u_3\}, L_{out}(u_3) = \{u_2, u_3\}$
u4	$L_{in}(u_4) = \{u_2, u_3\}, L_{out}(u_4) = \{\}$

$$\forall u_i, u_j \in V(G), u_i \rightarrow u_j \Leftrightarrow |L_{out}(u_i) \cap L_{in}(u_j)| > 0$$

$$e.g. \ u_1 \rightarrow u_4 \Leftrightarrow |\{L_{out}(u_1) \cap L_{in}(u_4)\}| = |\{u_2, u_3\}| > 0$$

示例



如何计算2-Hop覆盖?

2-hop Cover for hop u_2 .

for all $v \in C_{in}(u)$: $L_{out}(v) := L_{out}(v) \cup \{u\}$

for all $v \in C_{out}(u)$: $L_{in}(v) := L_{in}(v) \cup \{u\}$

问题定义



2-Hop Labeling的目标在于找出一个最优的编码策略, 使得如下定义的编码代价最小

$$\begin{aligned} LabelSize &= \sum_{u_i \in V(G)} (|L_{in}(u_i)| + |L_{out}(u_i)|) \\ &= \sum_{w_j \in hops} (|C_{in}(w_j)| + |C_{out}(w_j)|) \end{aligned}$$

可以证明, 这个问题能转化为集合覆盖问题, 进而难以找出多项式时间的算法

近似2-Hop编码求解算法



第一步，计算出所有点对之间的最短路径，形成集合T

第二步，挑选一个点w来最大化如下公式：

$$r(w) = \frac{|S(C_{in}(w), w, C_{out}(w)) \cap T'|}{|C_{in}(w)| + |C_{out}(w)|}$$

其中T'表示T中还没有被覆盖的路径

第三步， $T = T - T'$ ，然后迭代执行第1、2步直到T为空

标签限制的可达性查询



Lucien D. J. Valstar, George H. L. Fletcher, Yuichi Yoshida.

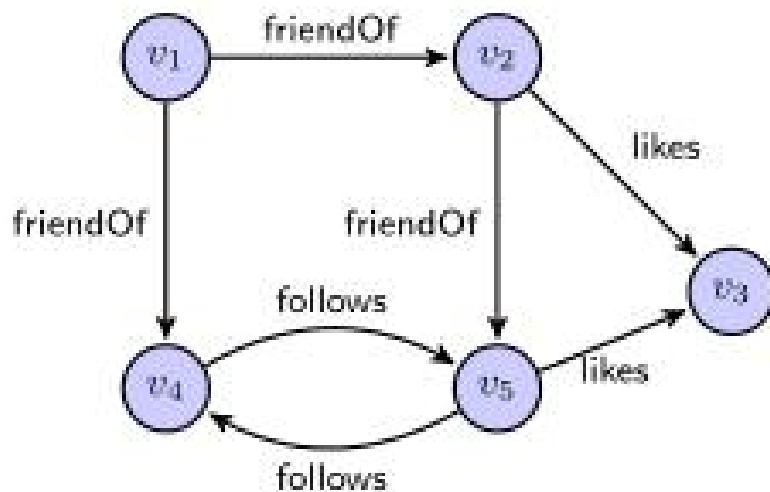
Landmark Indexing for Evaluation of Label-Constrained Reachability

Queries. SIGMOD Conference 2017: 345-358

问题定义



给定图 G 的顶点 s 和 t 以及 G 的所有边标签 L 集合的子集 L' ，仅使用 L' 中有标签的边来确定 G 中是否存在从 s 到 t 的路径。



查询($v_1, v_5, \{\text{friendOf}\}$), 返回True;
查询($v_1, v_3, \{\text{friendOf}\}$), 返回False;

基本思路



离线阶段: 给定输入图 $G = (V, E, \mathcal{L})$, 对于每个顶点 v , 如果存在从 v 到 w 的 L' 路径, 则每个 (w, L') 对存储到 v 的索引中

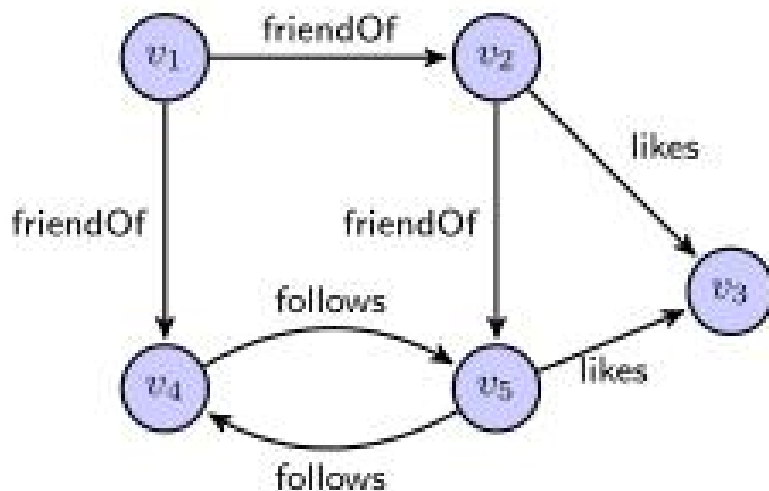
在线阶段: 通过检查对 (w, L') 是否在 v 的索引中来回答任何 LCR 查询 (v, w, L')

优化策略



对于点 v ，只存储 (w, L'') ，使得 L'' 是连接 v 到 w 的最小标签集。

查询 (v, w, L') 且 L'' 是 L' 子集，则返回True



查询 $(v_1, v_4, \{\text{friendOf}, \text{follows}\})$ 返回
True，标签集 $\{\text{friendOf}, \text{follows}\}$ 不
是最小标签集合，因为查询 $(v_1, v_4,$
 $\{\text{friendOf}\})$ 也返回True

方法概述



离线阶段：只为少量顶点（称为Landmark）构造索引

在线阶段：给定一个查询 (s, t, L) ，我们从 s 中执行BFS（考虑标签）。当我们找到一个为其构造索引的地标 x 时，我们使用它立即获得答案。



湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——